Abstract Data Type

queue

preliminaries

- a word on ADT
  - different implementations successfully solving the same problem
- applications of lists
  - grocery list
  - to-do list
  - polynomial expression
  - unbounded integers
  - text



preliminaries

- some problems require a list but only access/manipulate the first element and last element
  - lining up in the cashier
  - lining up for relief goods
  - lining up in a bus station
  - ??



queue

- first-in-first-out ADT (FIFO)
  - removes items in the order they were added
- operations
  - enqueue
  - dequeue
  - front
  - rear

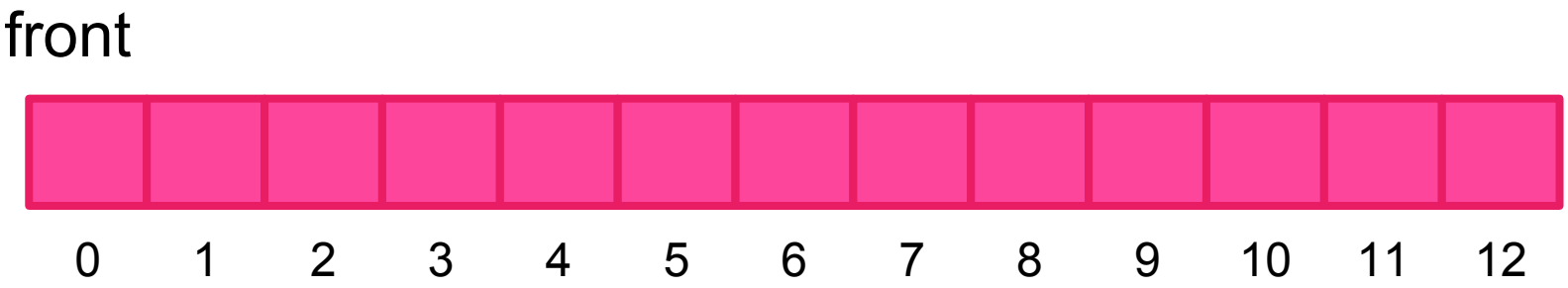


queue

- implementations
  - array
    - regular
    - circular
  - vector
  - linked list
    - singly-linked
    - doubly-linked

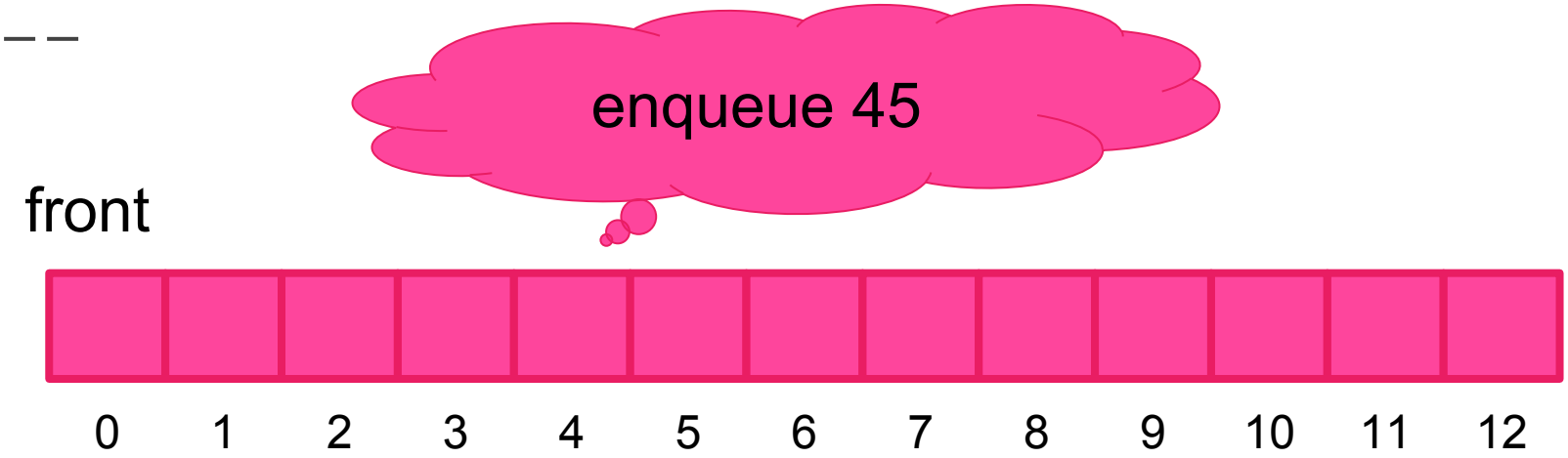


array implementation

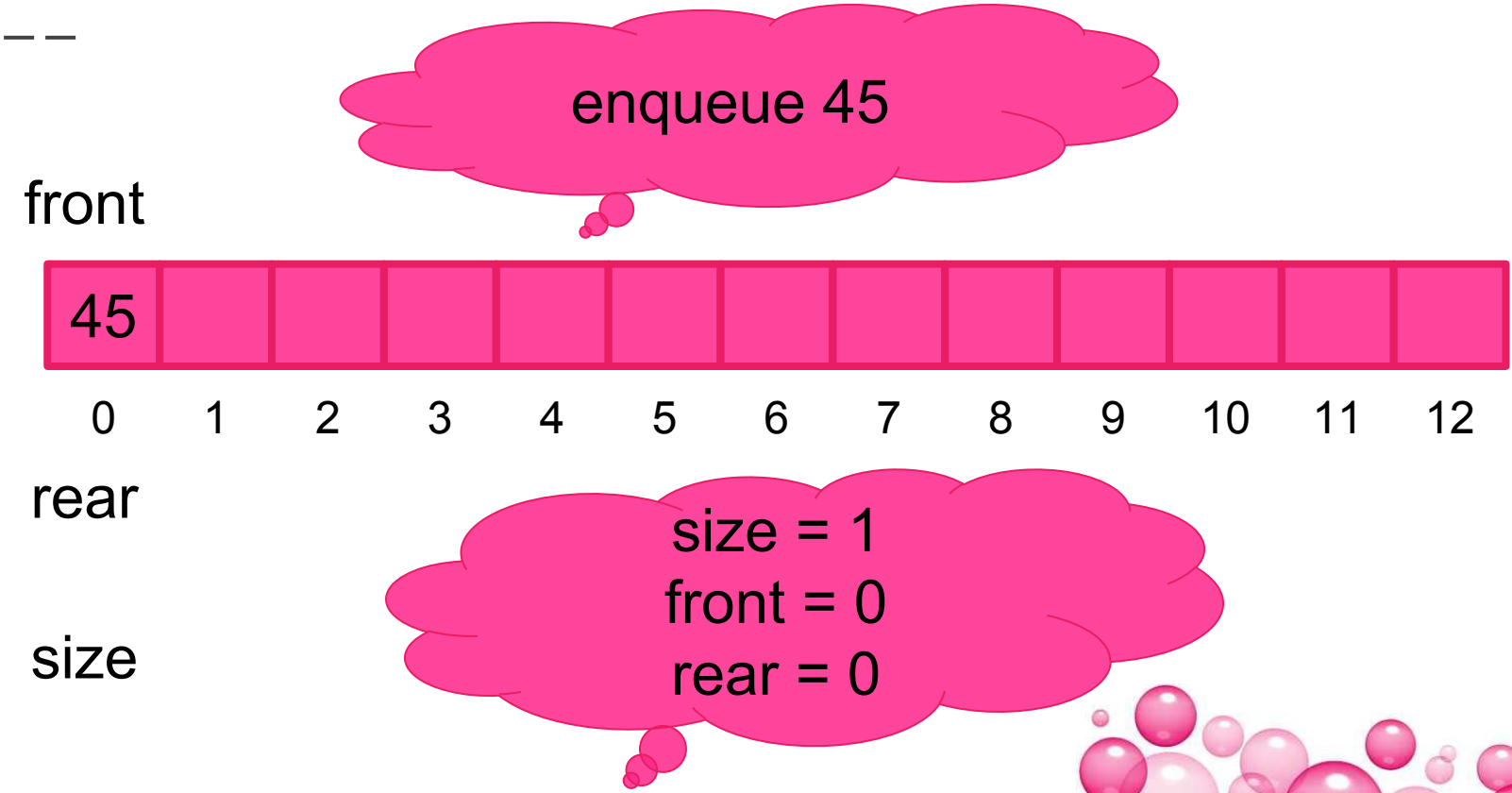


array implementation - enqueue (empty case)

– – –



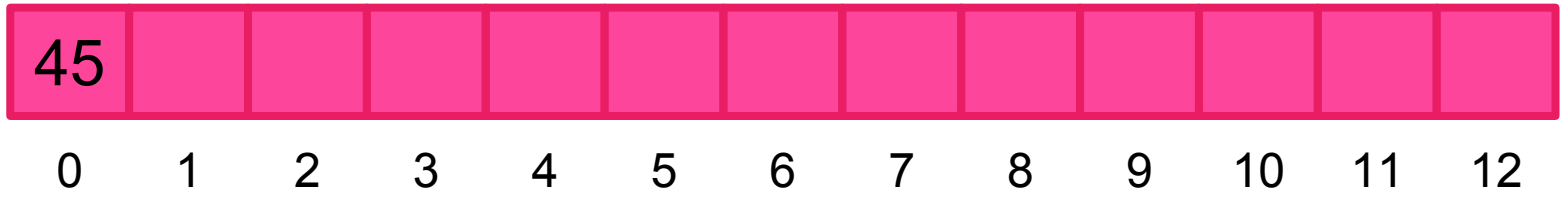
array implementation - enqueue (empty case)



array implementation - enqueue

enqueue -3

front



rear

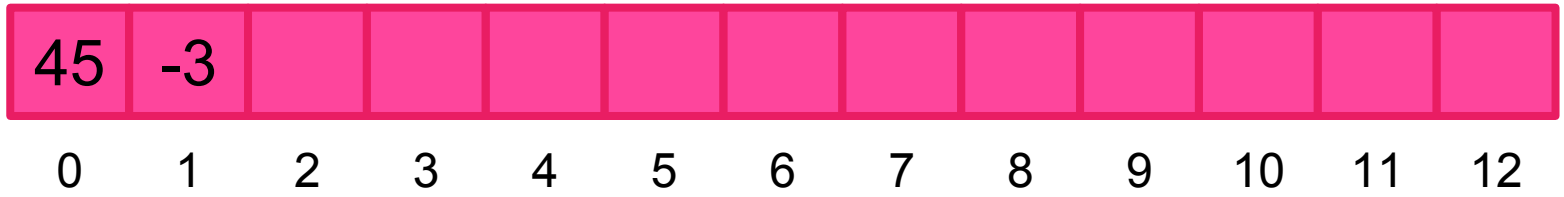
size

size = 1
front = 0
rear = 0

array implementation - enqueue

enqueue -3

front

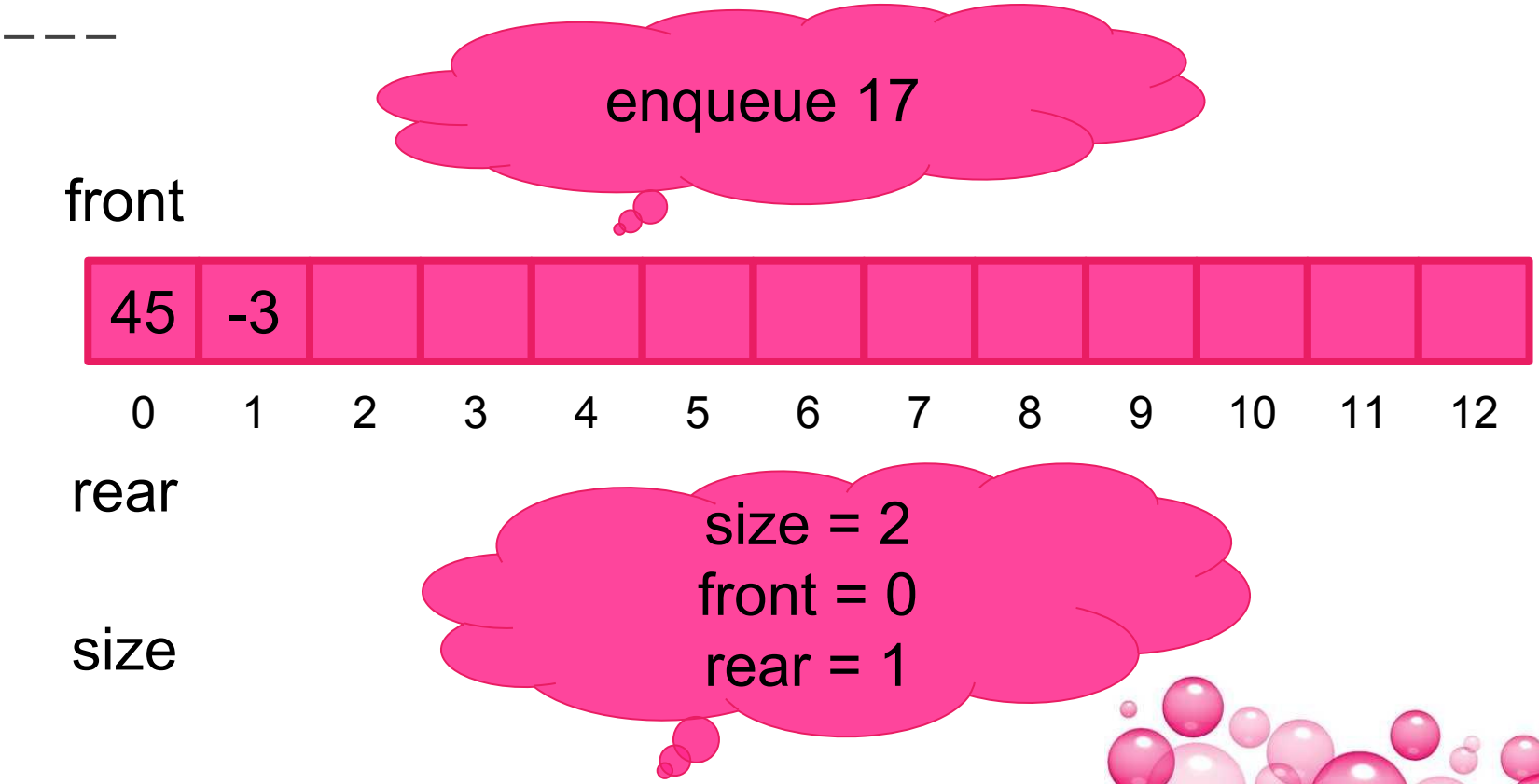


rear

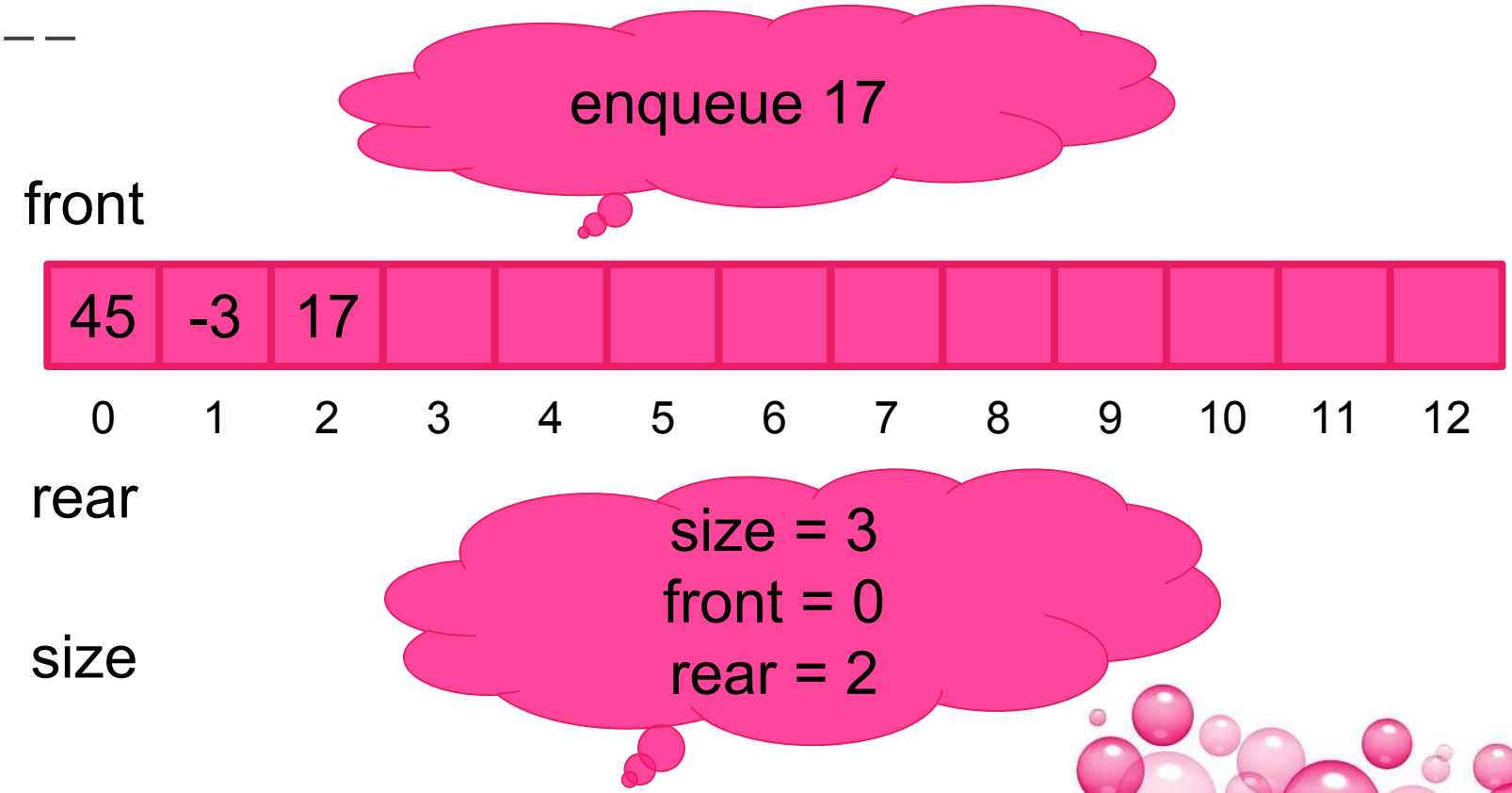
size

size = 2
front = 0
rear = 1

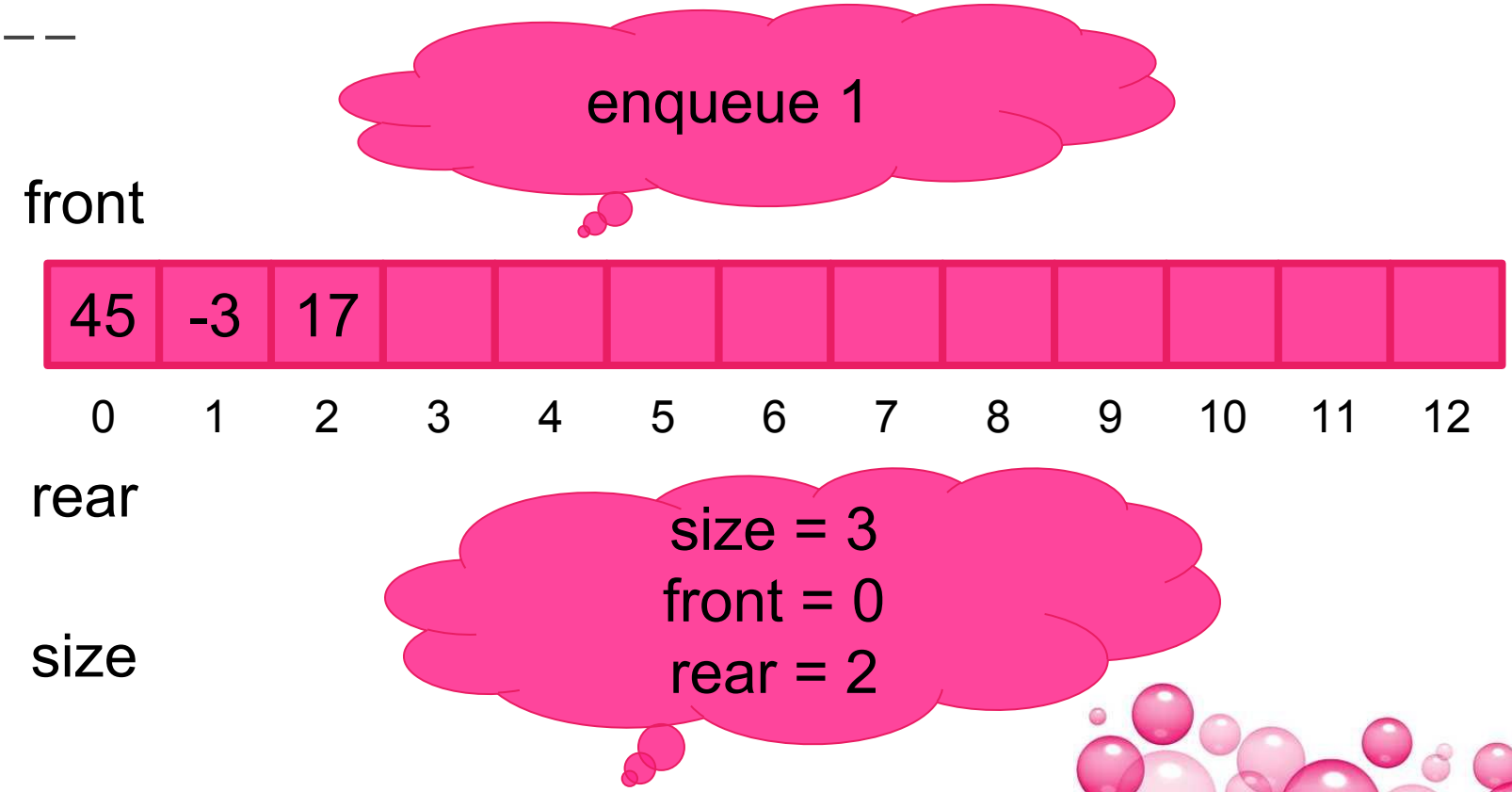
array implementation - enqueue



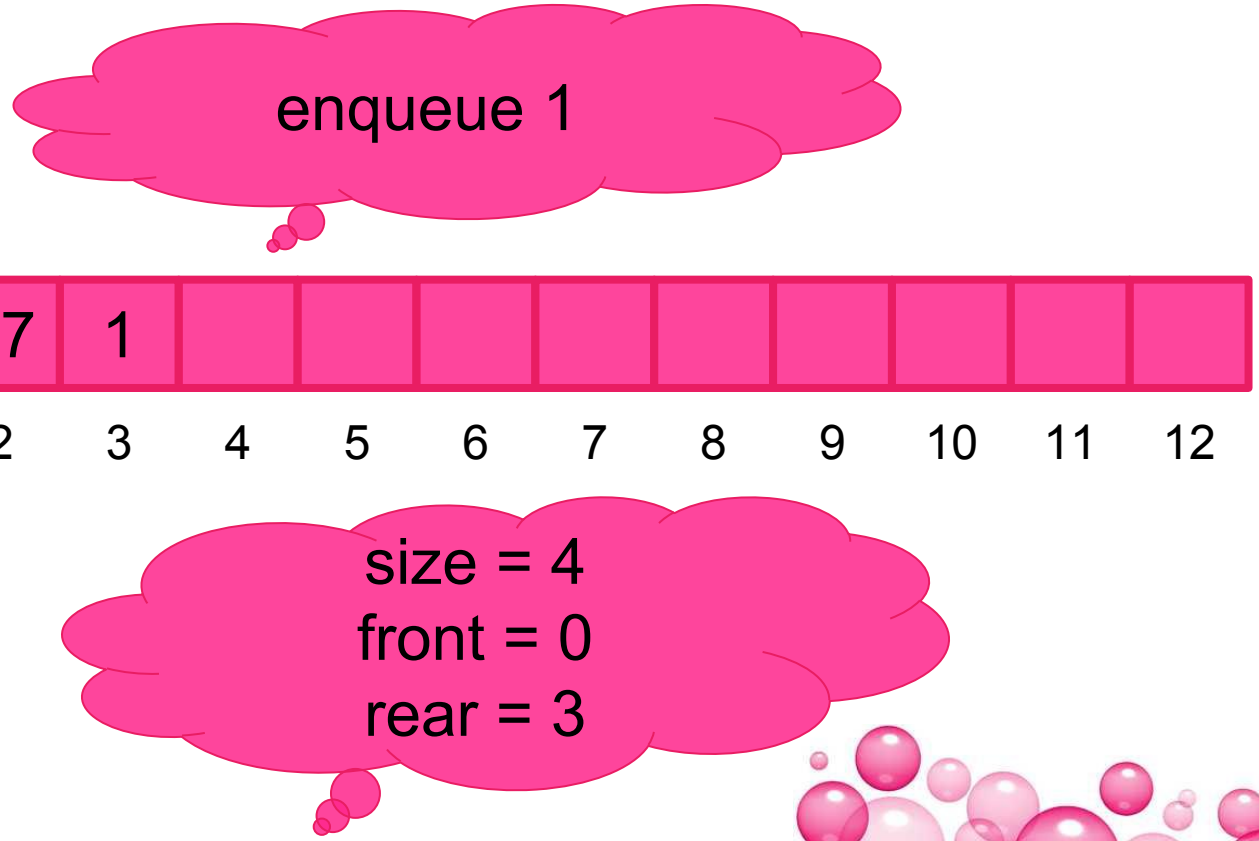
array implementation - enqueue



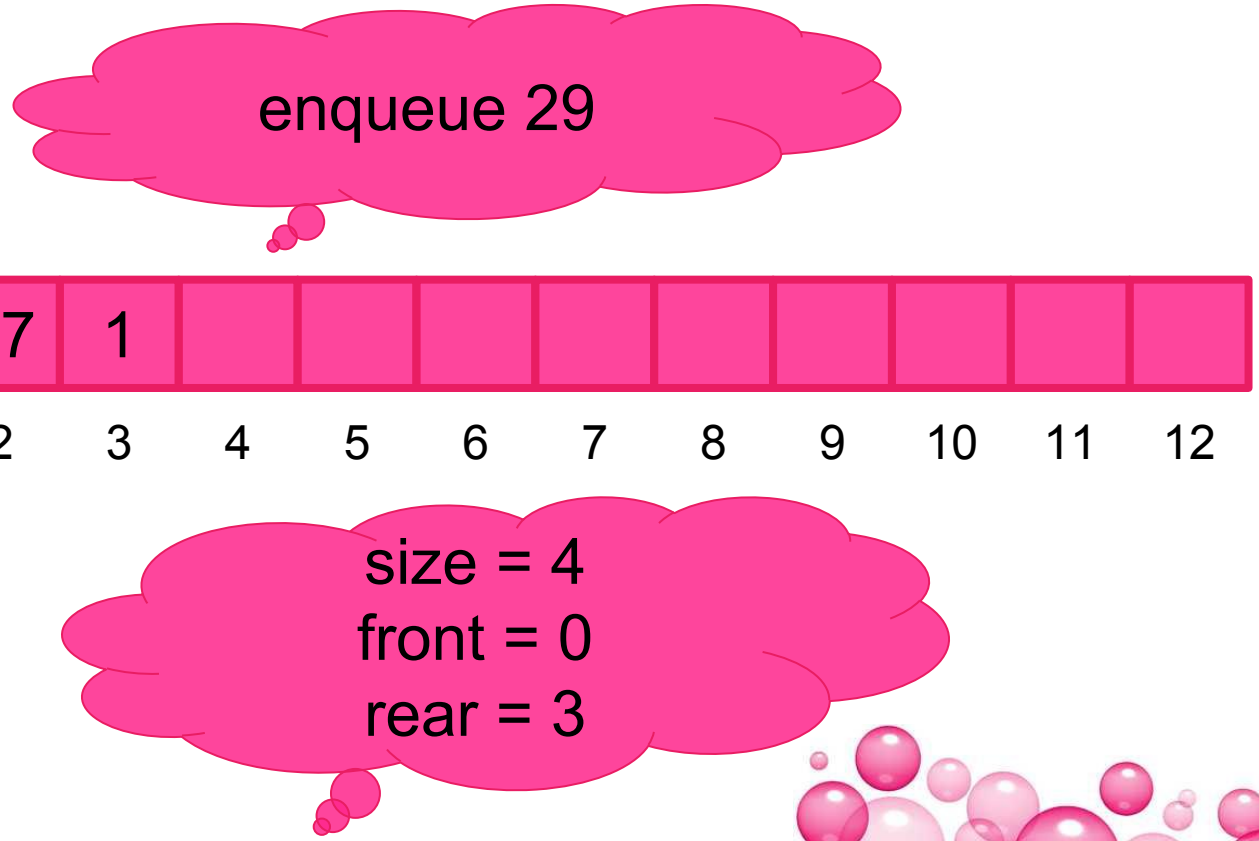
array implementation - enqueue



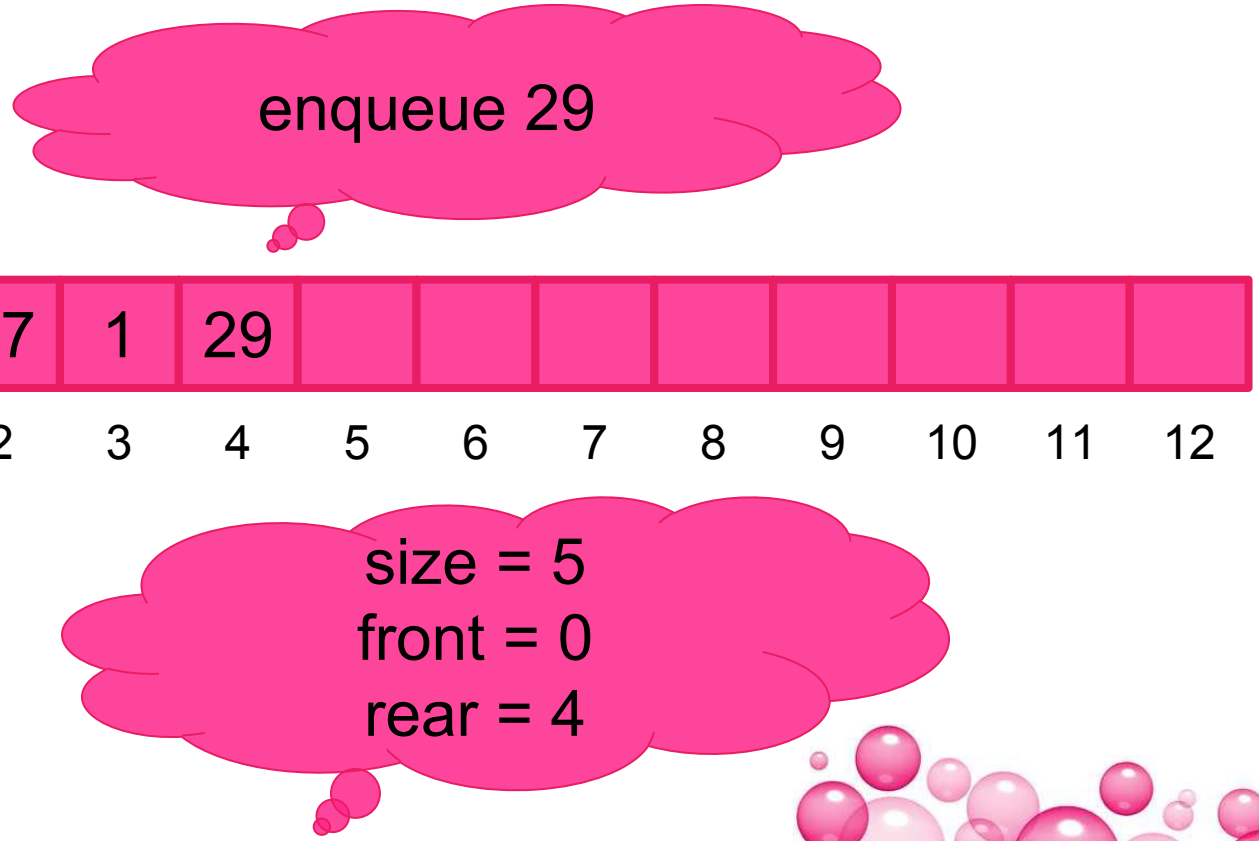
array implementation - enqueue



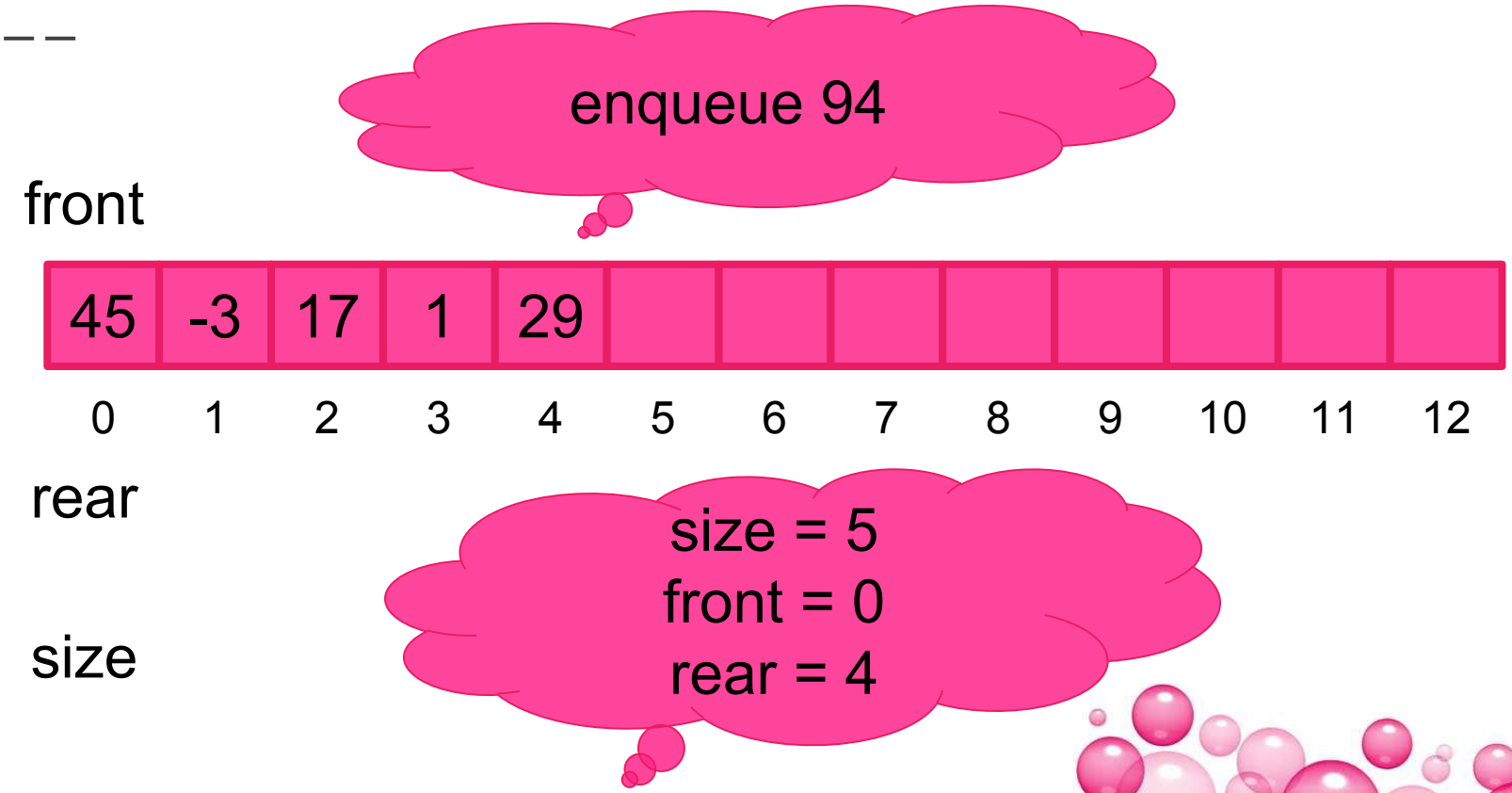
array implementation - enqueue



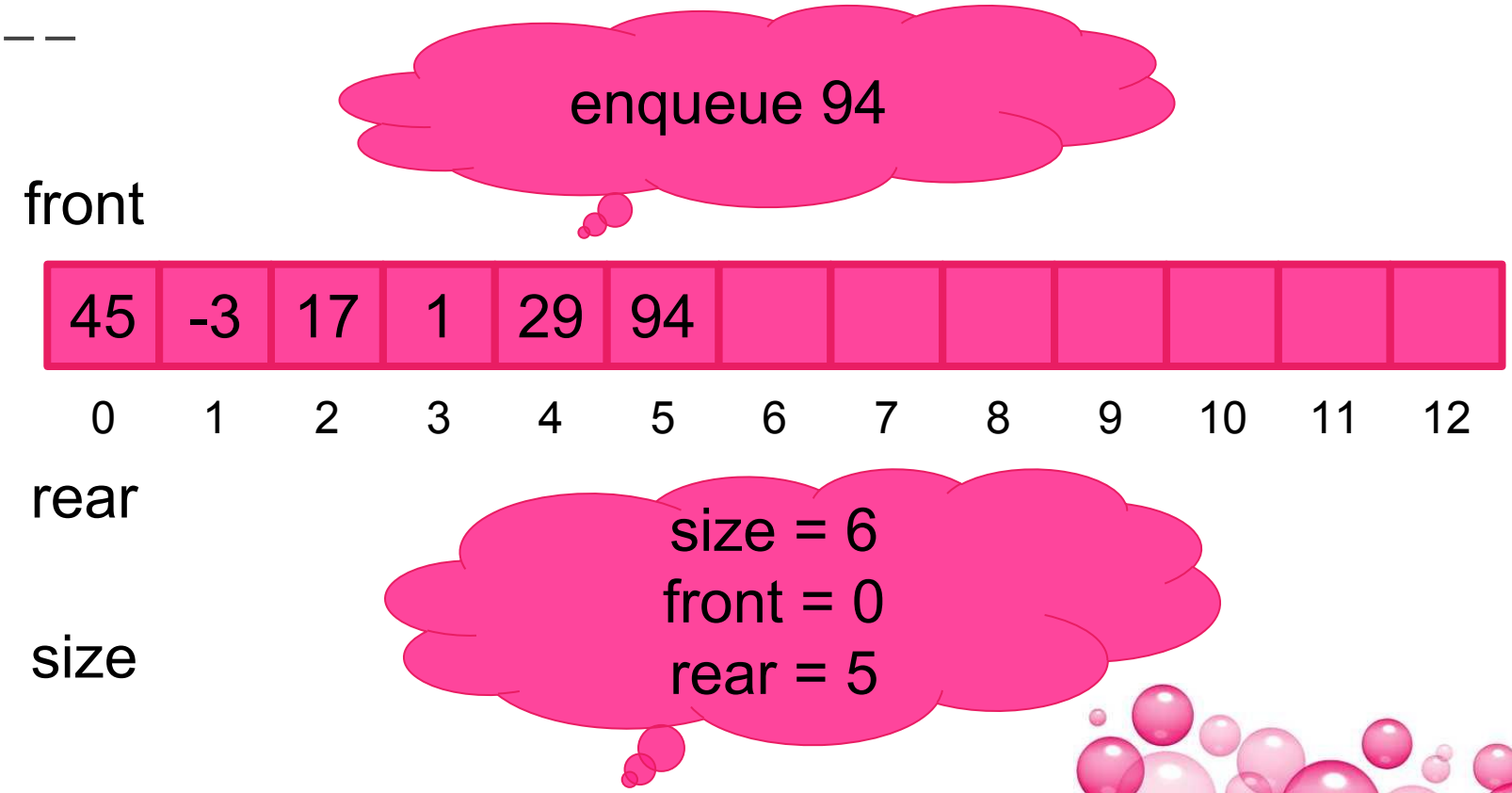
array implementation - enqueue



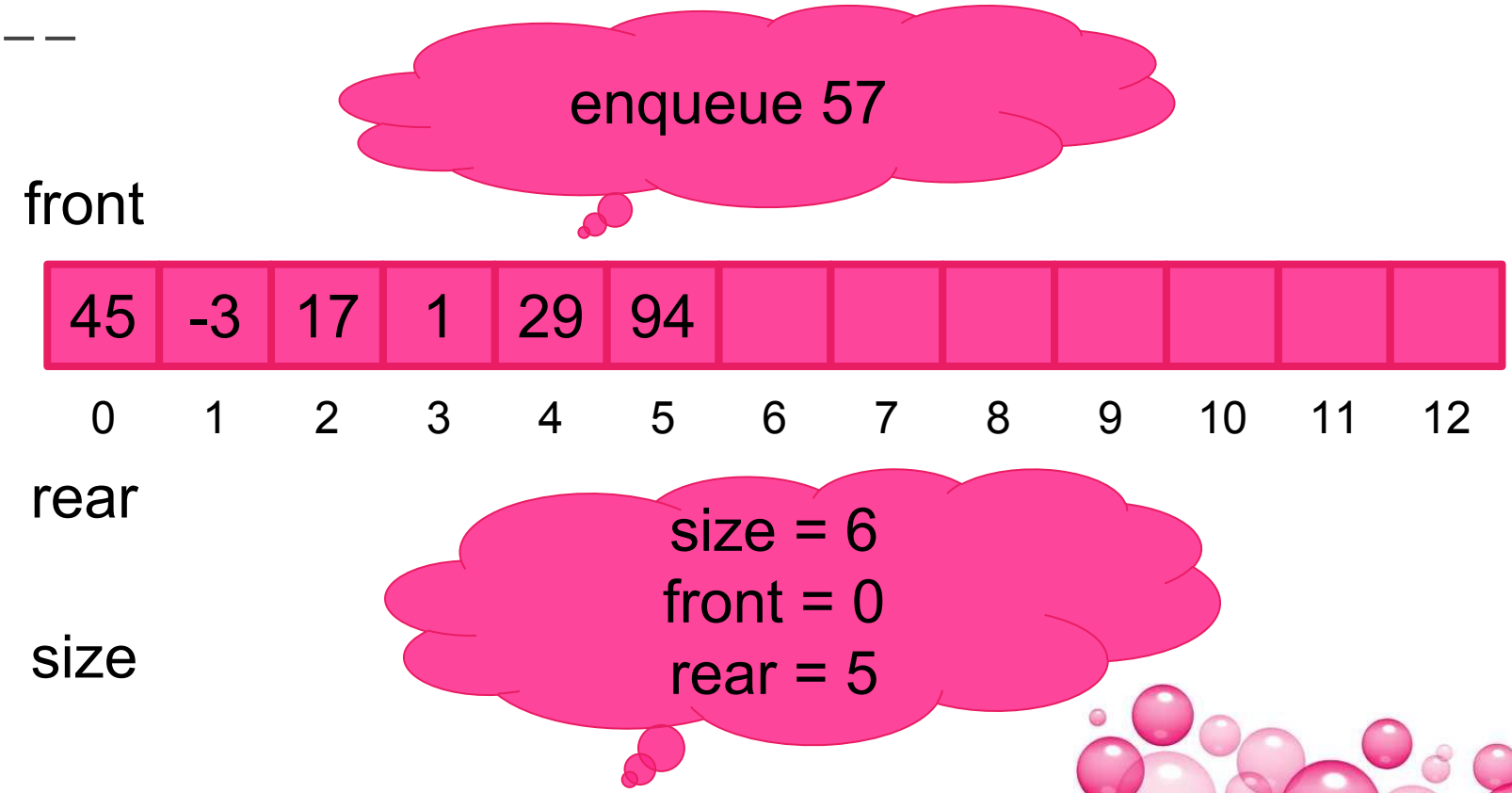
array implementation - enqueue



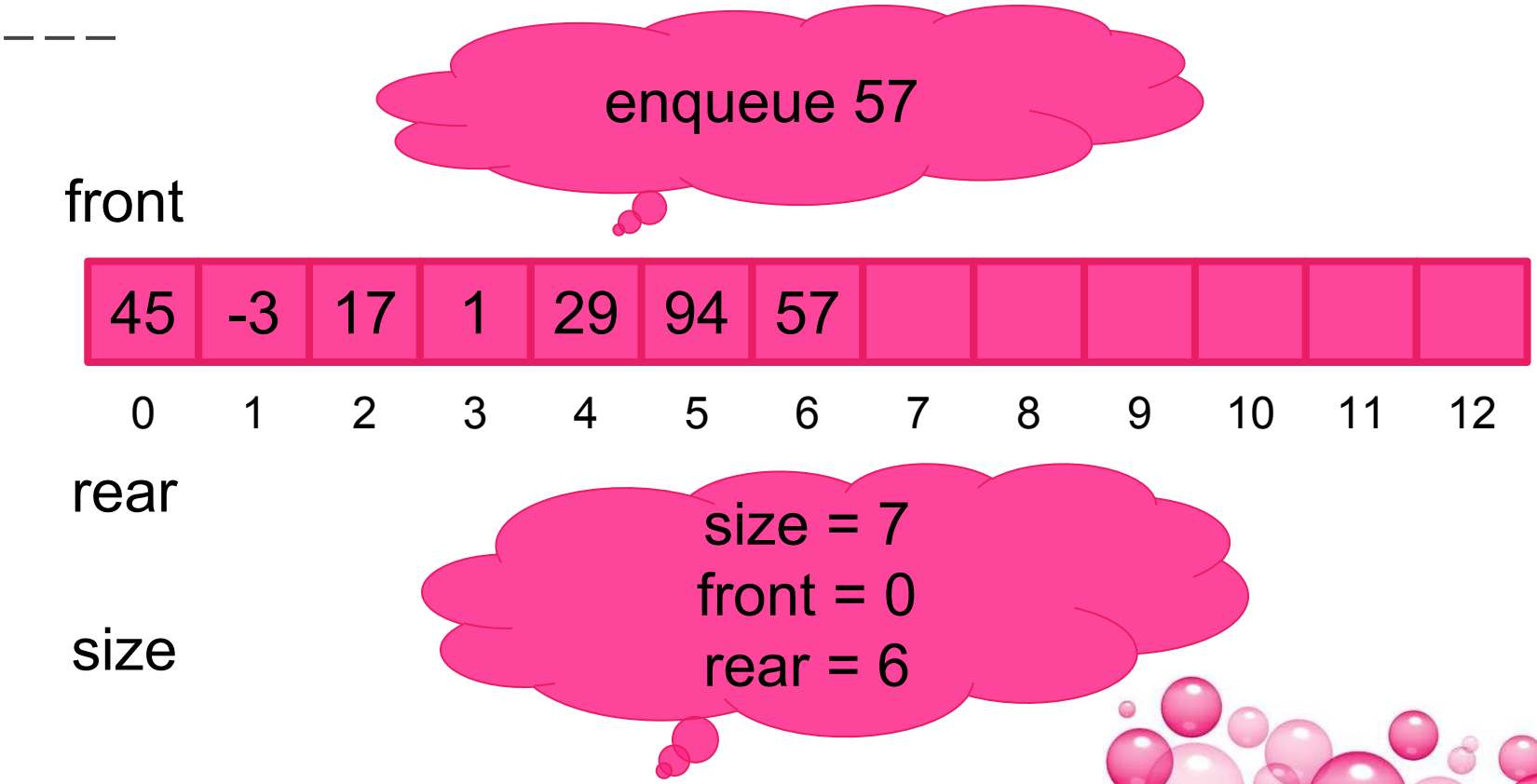
array implementation - enqueue



array implementation - enqueue

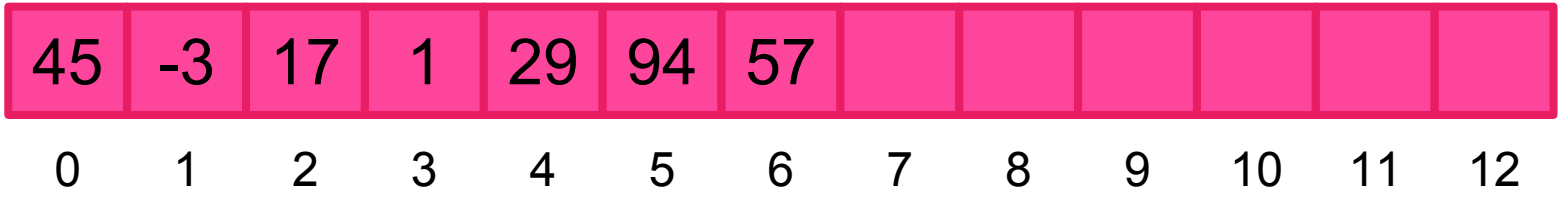


array implementation - enqueue



array implementation - dequeue

front



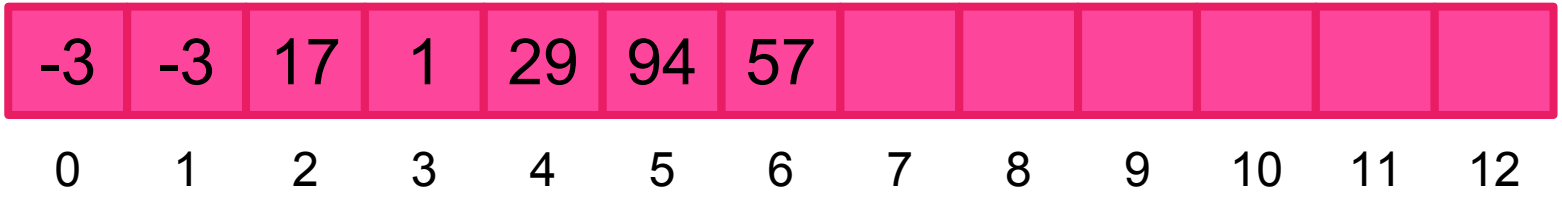
rear

size

size = 7
front = 0
rear = 6

array implementation - dequeue

front



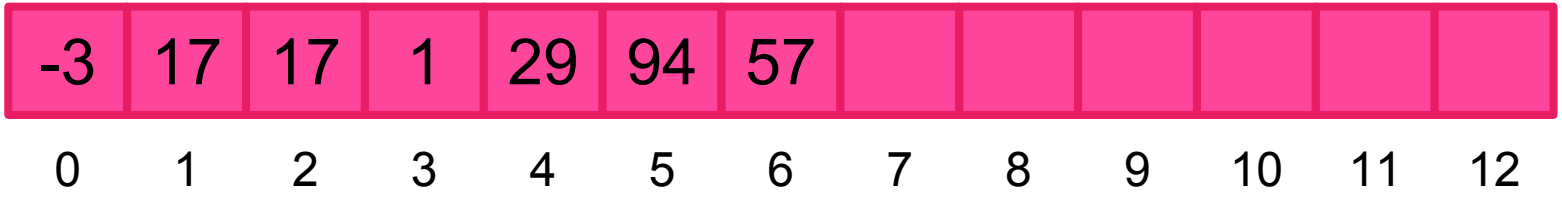
rear

size

size = 7
front = 0
rear = 6

array implementation - dequeue

front



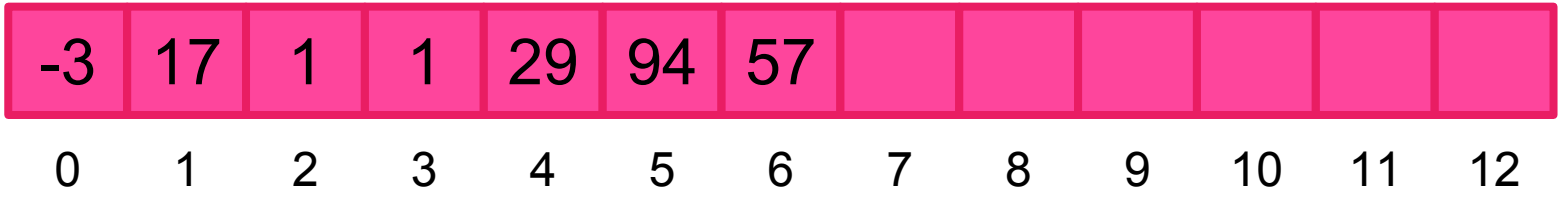
rear

size

size = 7
front = 0
rear = 6

array implementation - dequeue

front



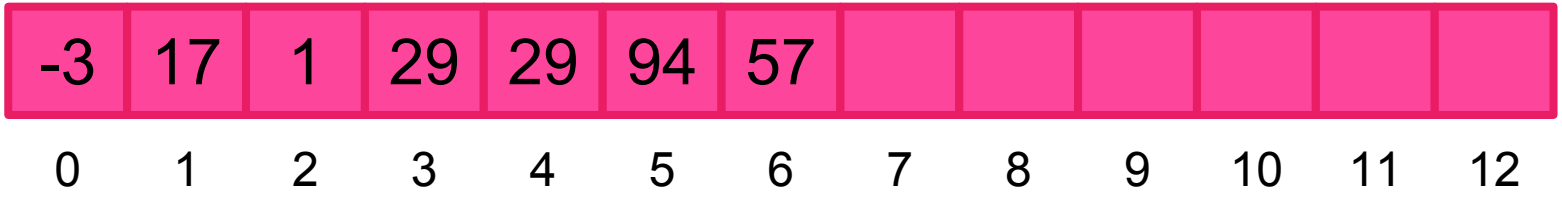
rear

size

size = 7
front = 0
rear = 6

array implementation - dequeue

front



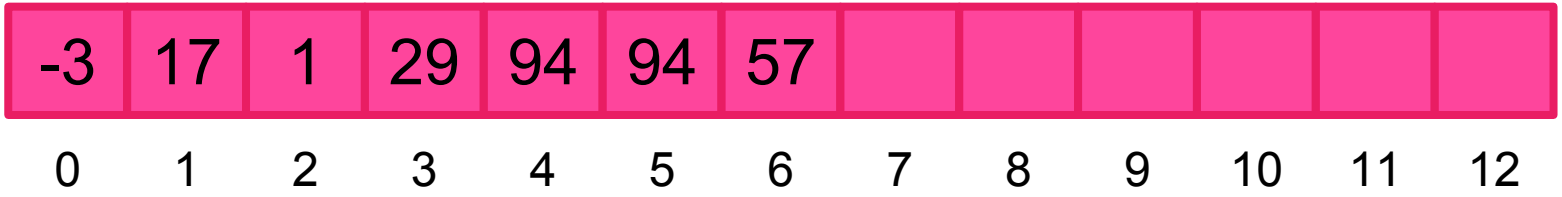
rear

size

size = 7
front = 0
rear = 6

array implementation - dequeue

front



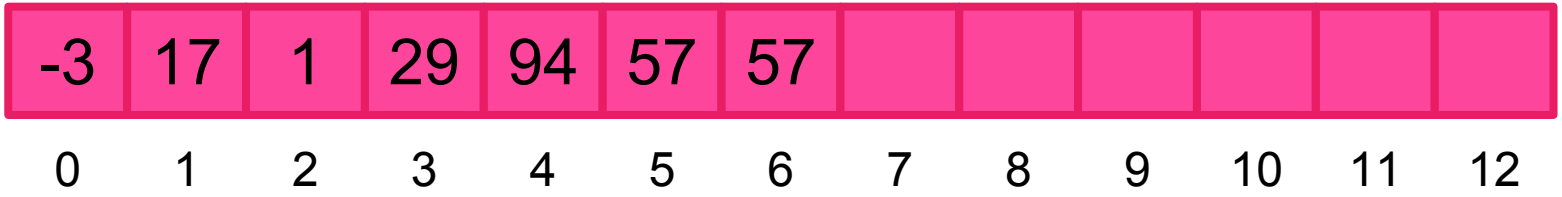
rear

size

size = 7
front = 0
rear = 6

array implementation - dequeue

front



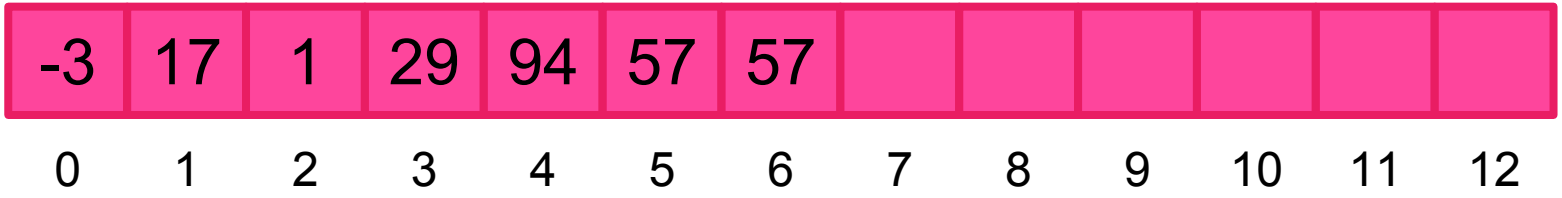
rear

size

size = 7
front = 0
rear = 6

array implementation - dequeue

front



rear

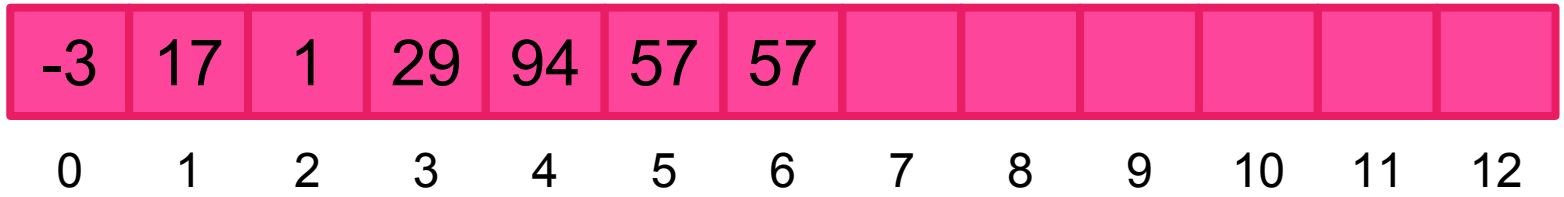
size

size = 6
front = 0
rear = 5

array implementation - enqueue

two 57s?

front

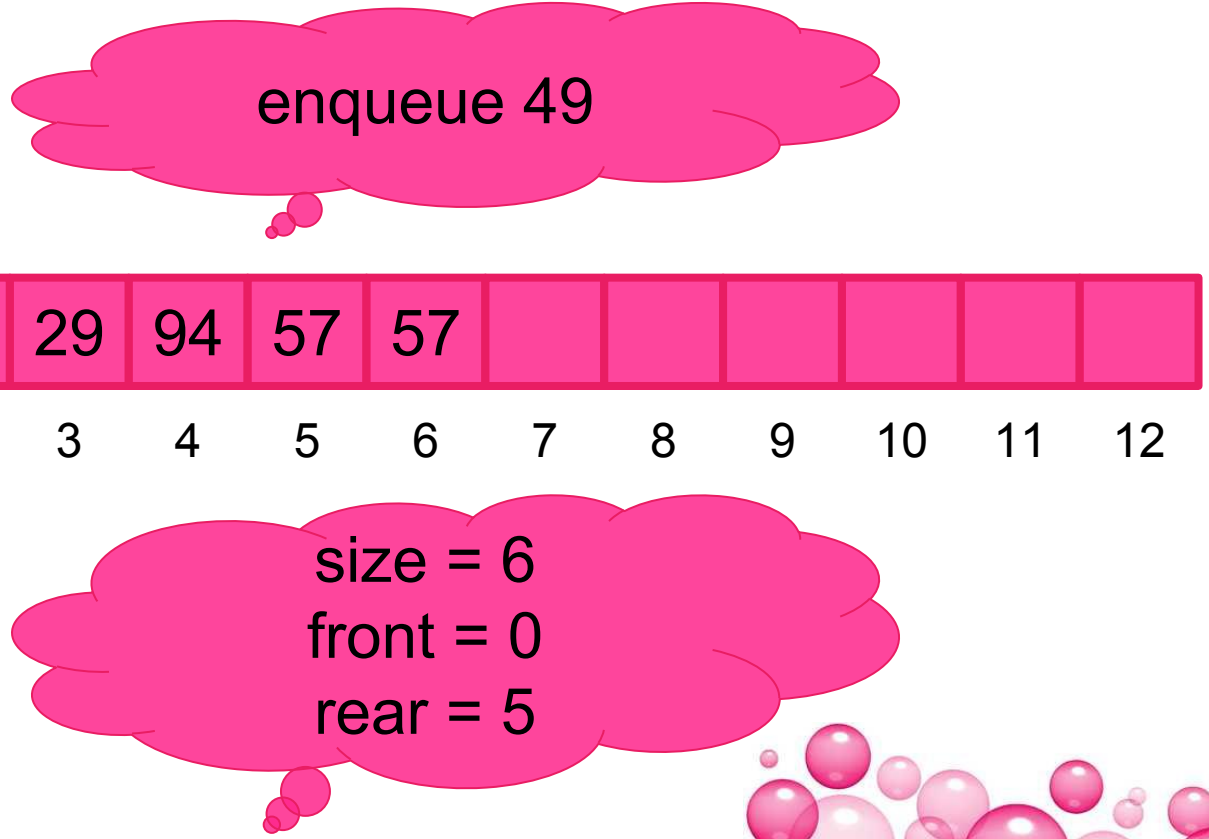


rear

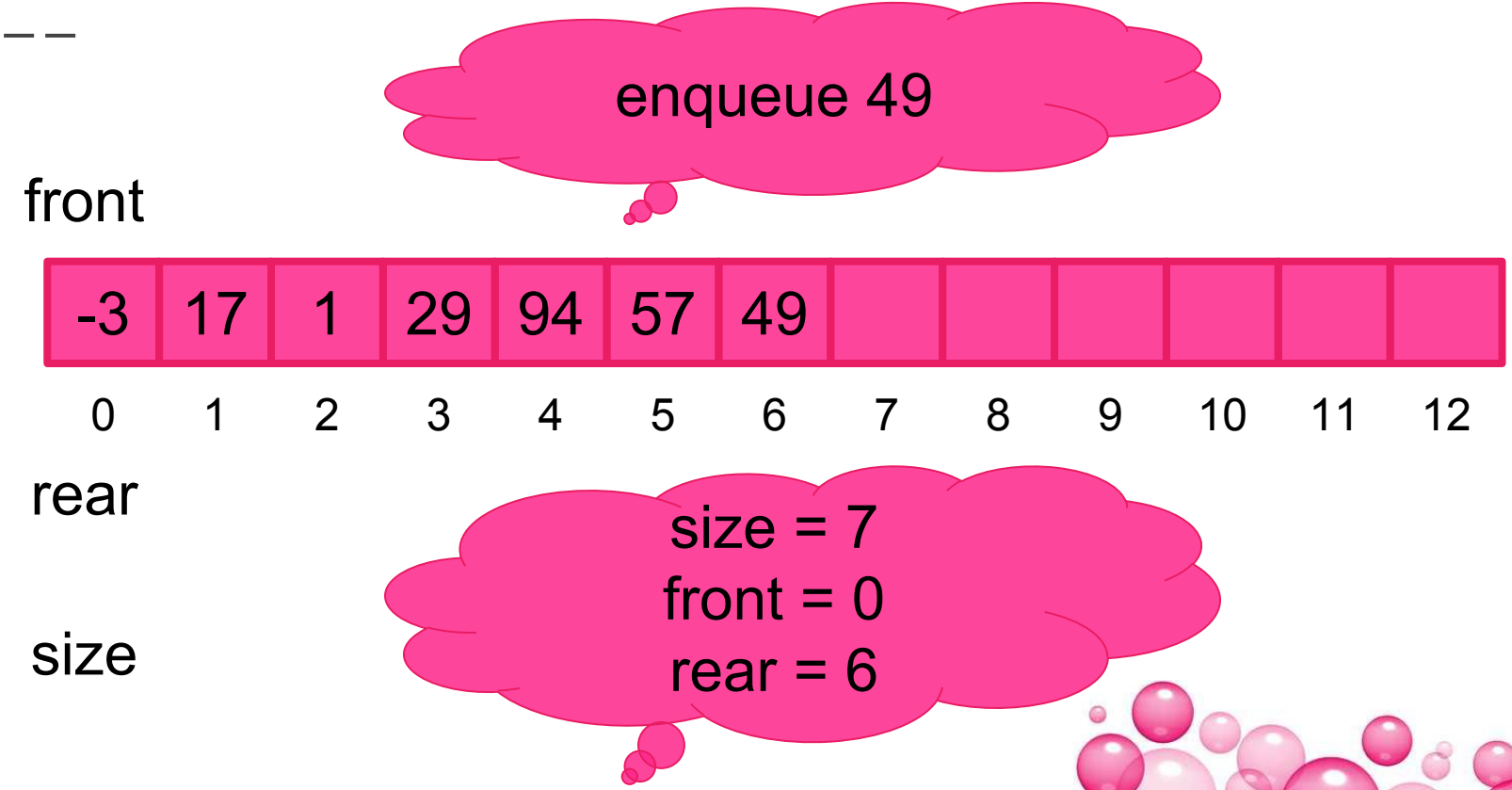
size

size = 6
front = 0
rear = 5

array implementation - enqueue



array implementation - enqueue



array implementation - data members

do we need front?

front



0 1 2 3 4 5 6 7 8 9 10 11 12

rear

size = 7

~~front = 0~~

rear = 6

size

array implementation - data members

do we need rear?

front



0 1 2 3 4 5 6 7 8 9 10 11 12

rear

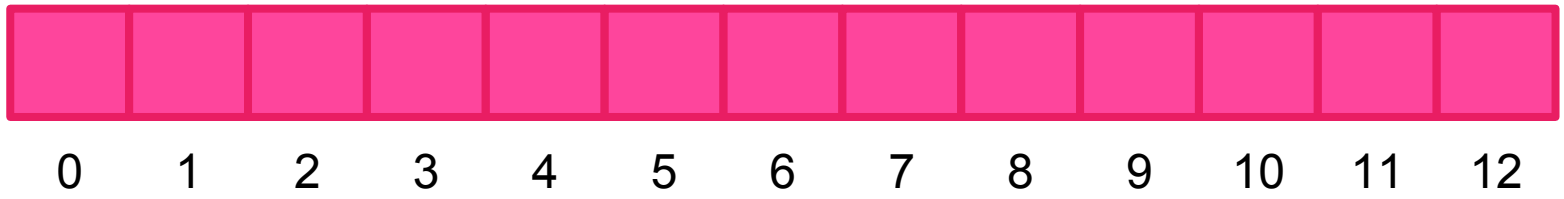
size = 7

~~front = 0~~

~~rear = 6~~

size

array implementation



size = 0

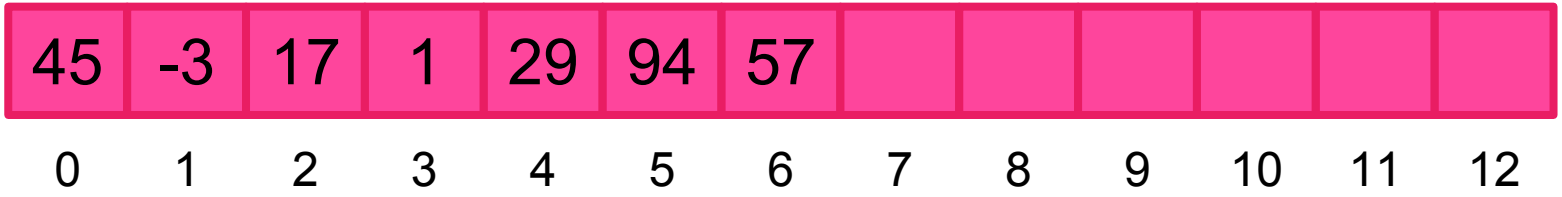
size

array implementation

operation	array
enqueue	O(1)
dequeue	O(n)
front	O(1)
rear	O(1)



array implementation - addressing linear dequeue

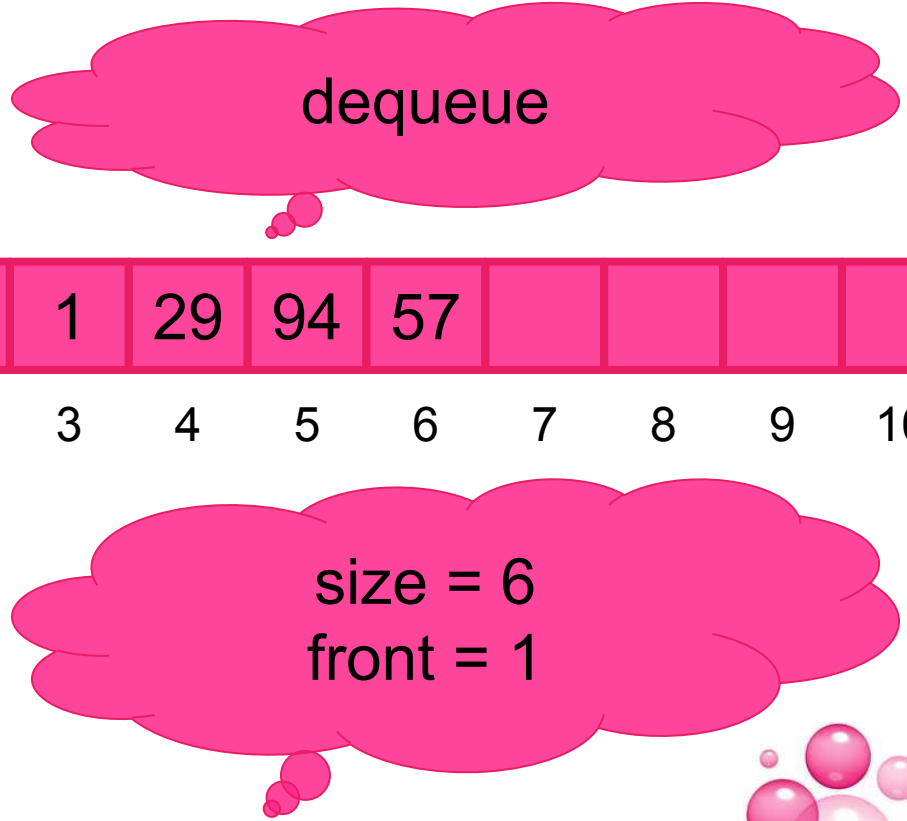


front

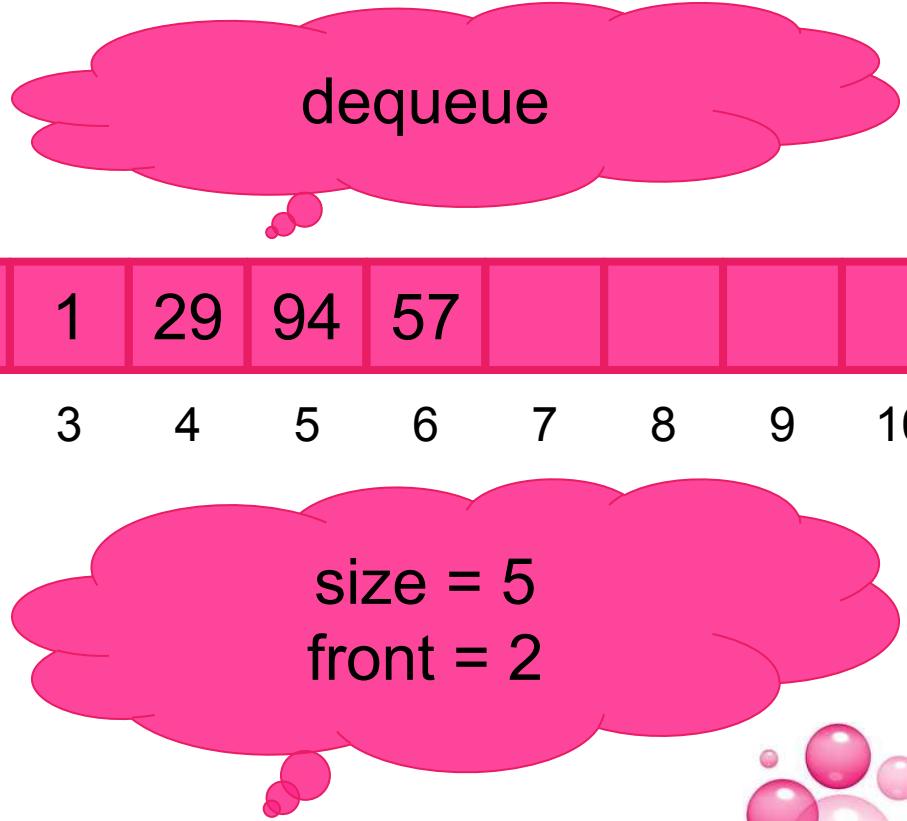
size

size = 7
front = 0

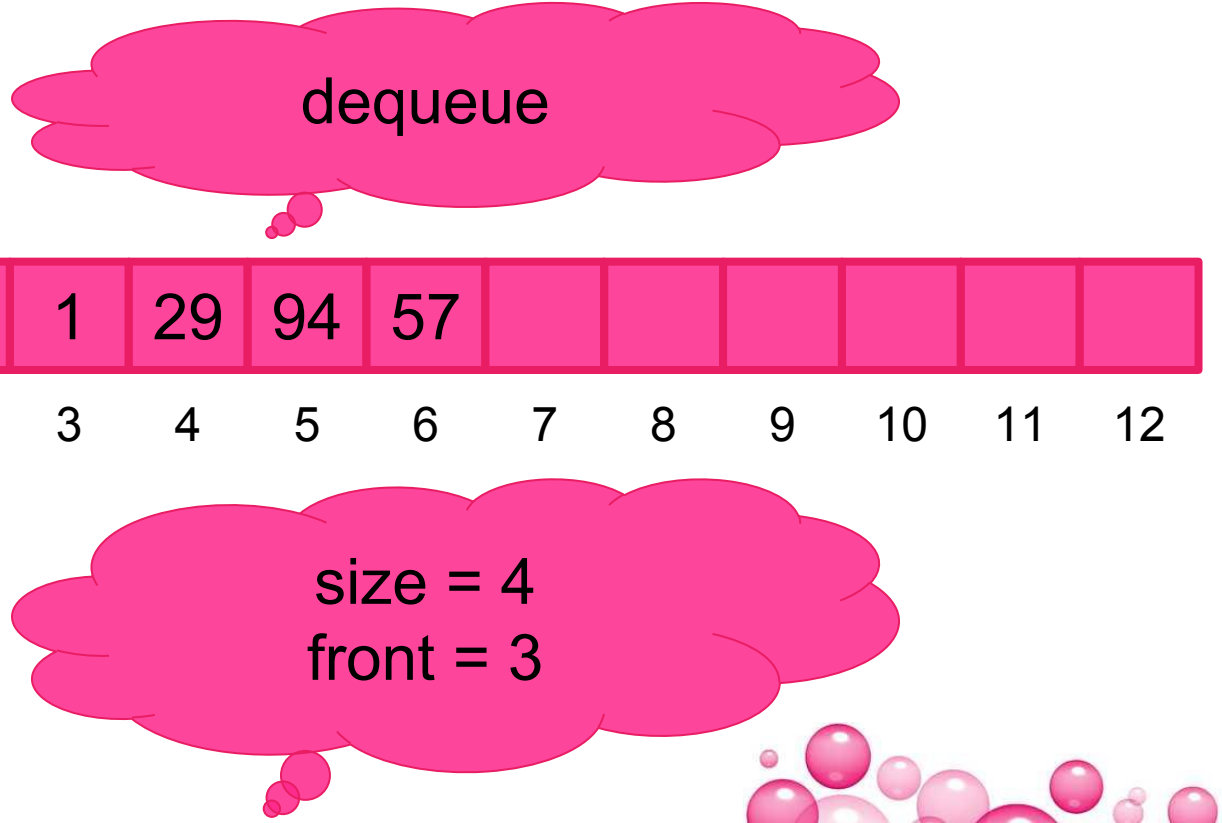
array implementation - addressing linear dequeue



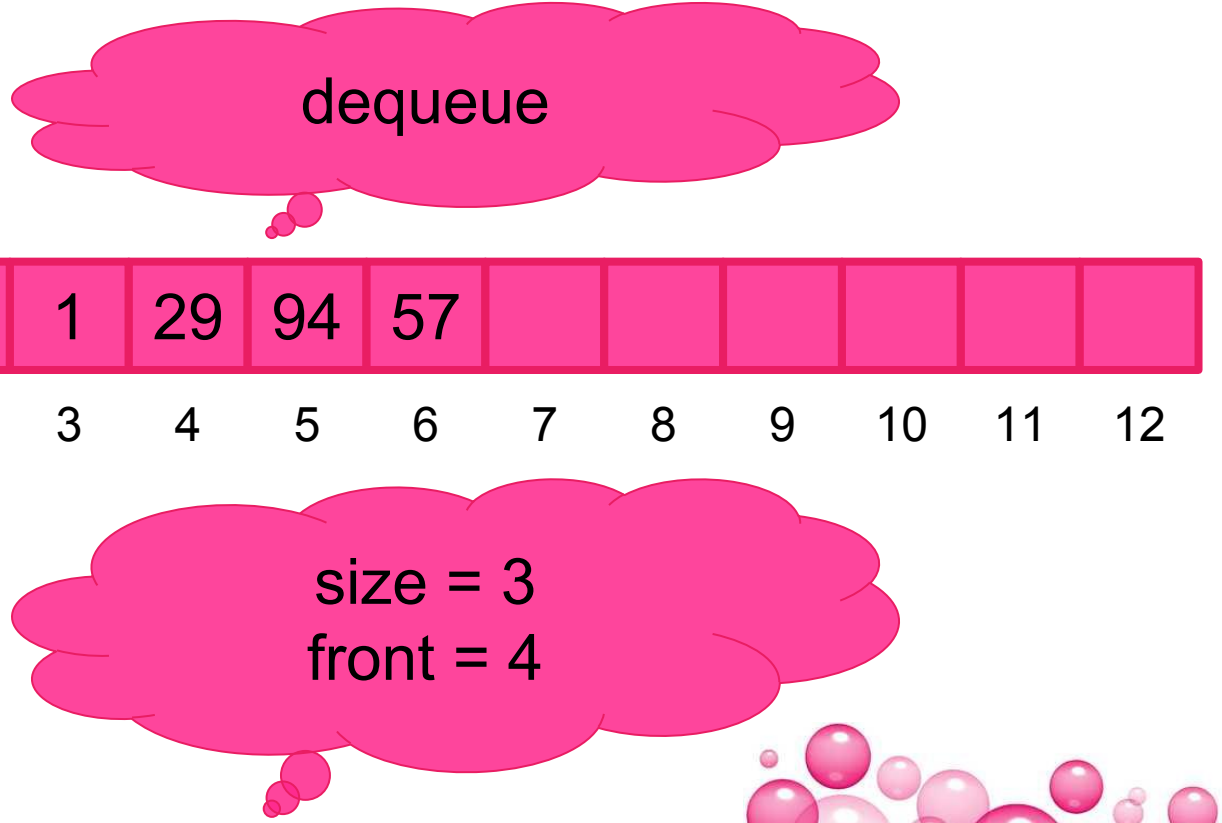
array implementation - addressing linear dequeue



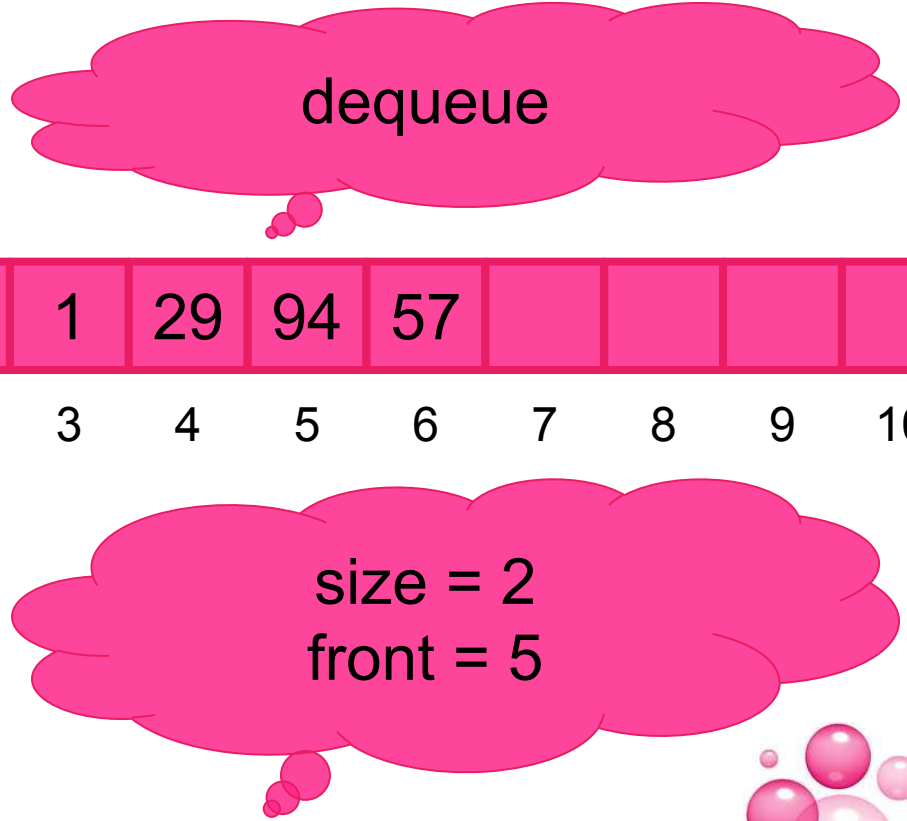
array implementation - addressing linear dequeue



array implementation - addressing linear dequeue



array implementation - addressing linear dequeue



array implementation - addressing linear dequeue

enqueue 36



0 1 2 3 4 5 6 7 8 9 10 11 12

front

size

size = 3
front = 5

array implementation - addressing linear dequeue

enqueue 78



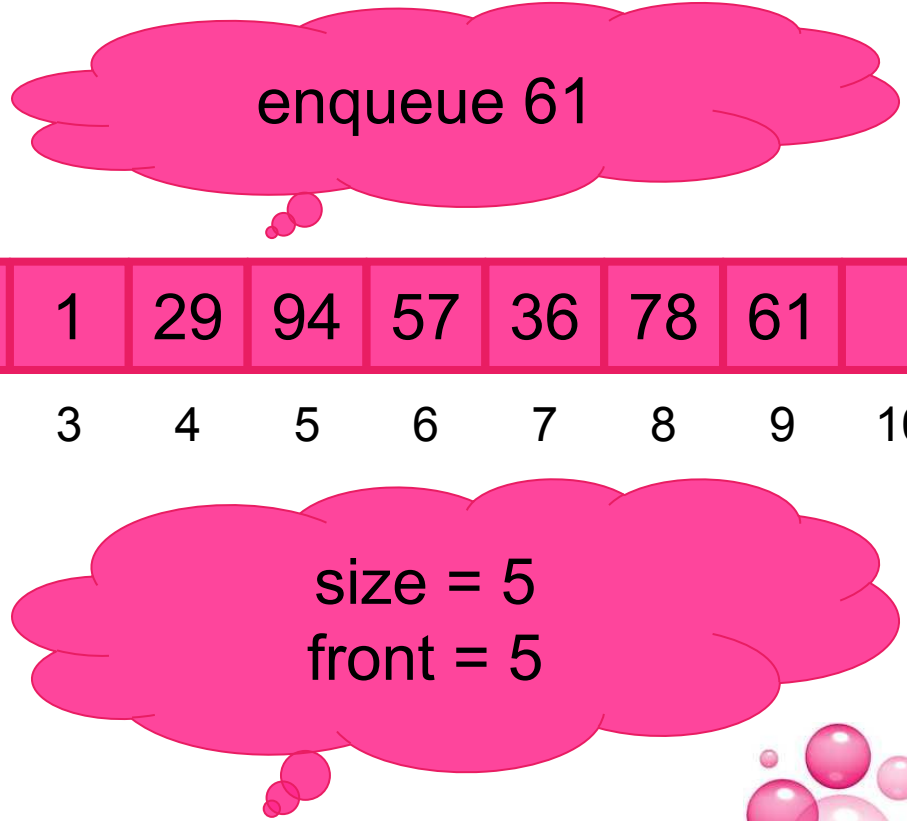
0 1 2 3 4 5 6 7 8 9 10 11 12

front

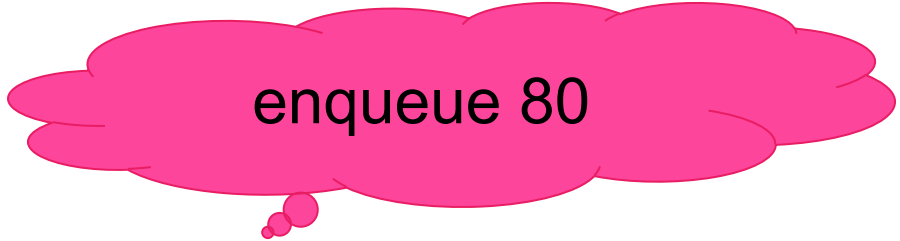
size

size = 4
front = 5

array implementation - addressing linear dequeue



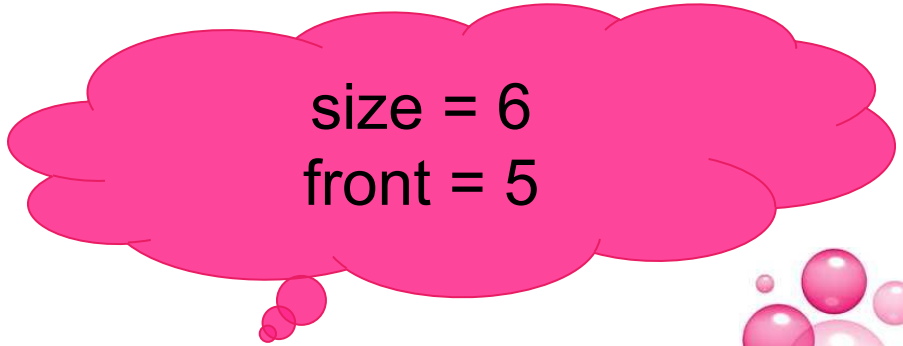
array implementation - addressing linear dequeue



45	-3	17	1	29	94	57	36	78	61	80		
0	1	2	3	4	5	6	7	8	9	10	11	12

front

size



array implementation - addressing linear dequeue

enqueue 5



0 1 2 3 4 5 6 7 8 9 10 11 12

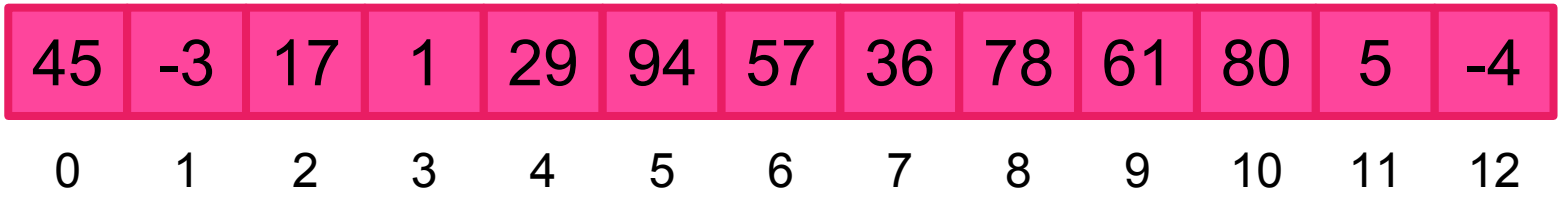
front

size

size = 7
front = 5

array implementation - addressing linear dequeue

enqueue -4



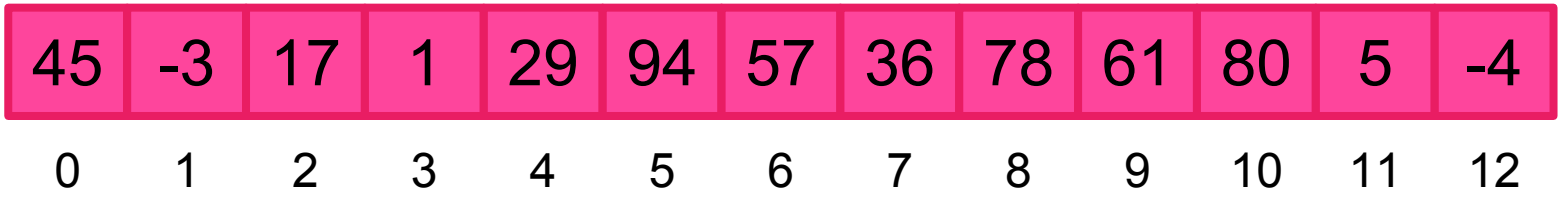
front

size

size = 8
front = 5

array implementation - addressing linear dequeue

enqueue 13

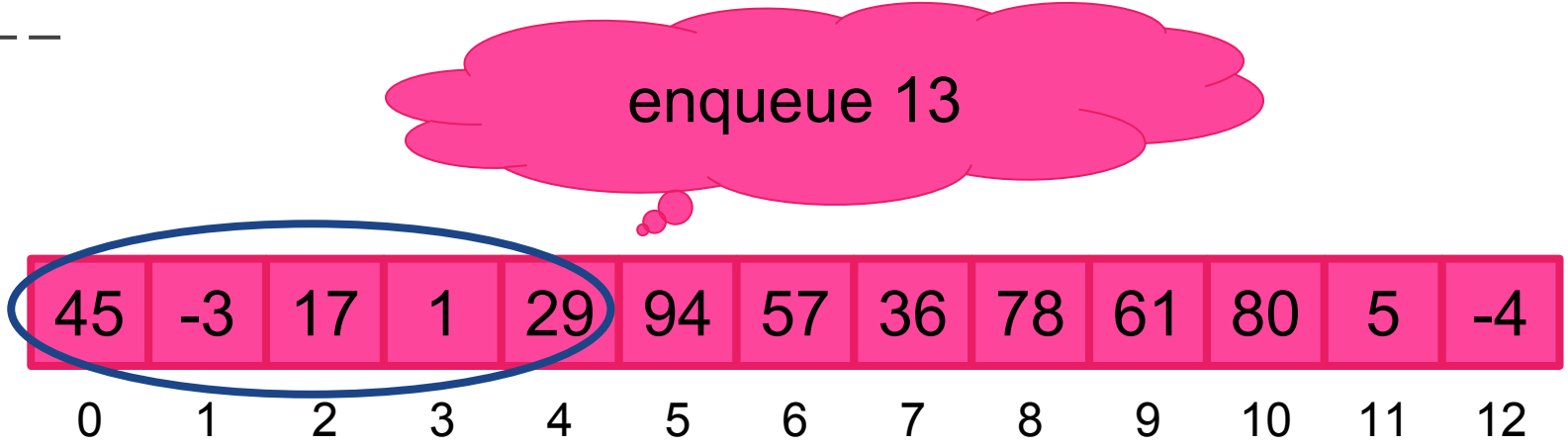


front

size

size = 8
front = 5

array implementation - addressing linear dequeue

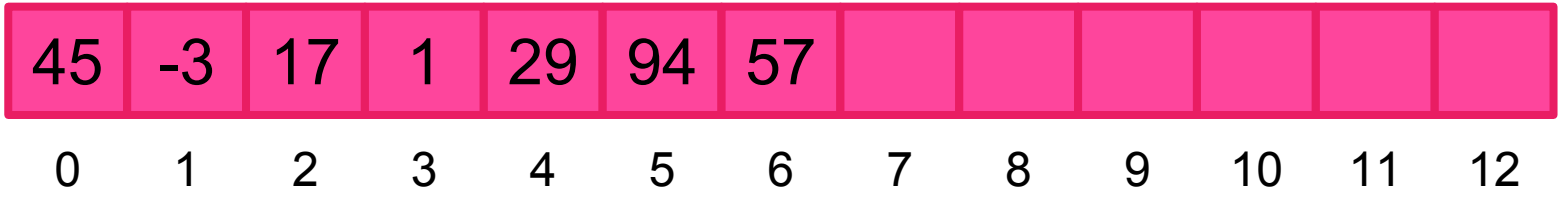


front

size

size = 8
front = 5

array implementation - addressing linear dequeue

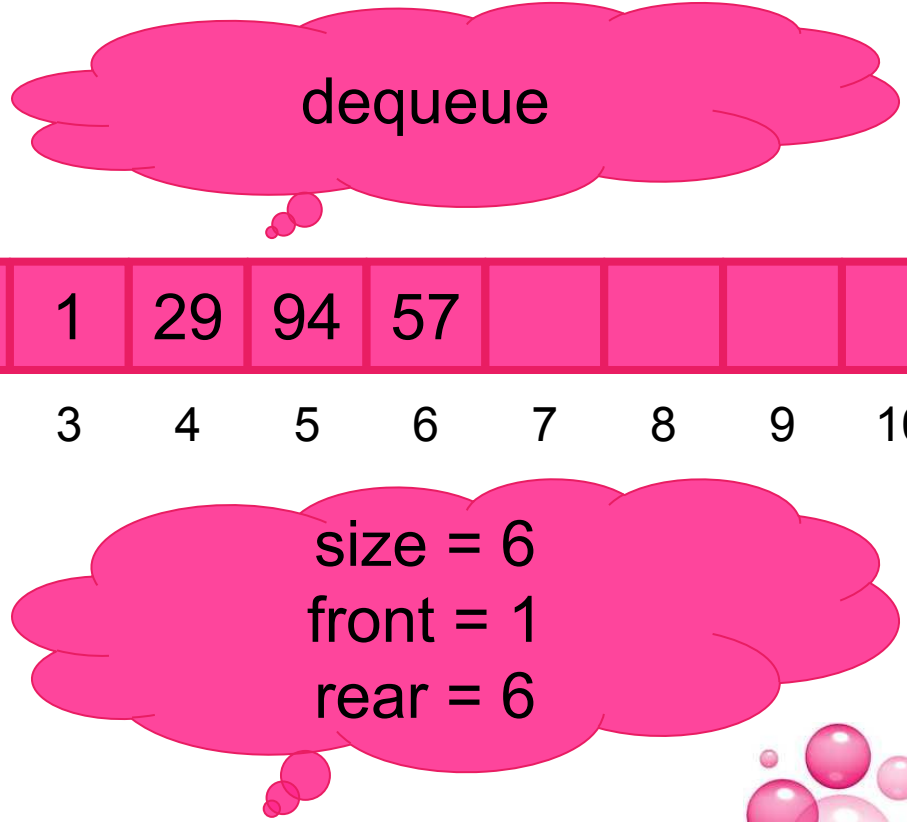


front

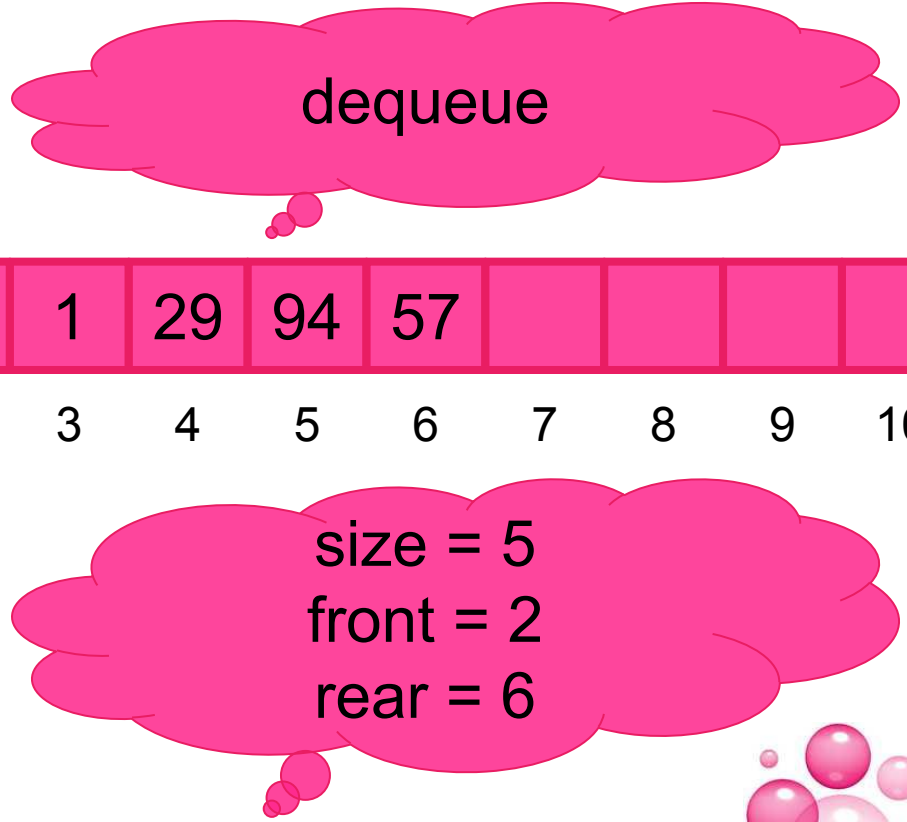
size

size = 7
front = 0
rear = 6

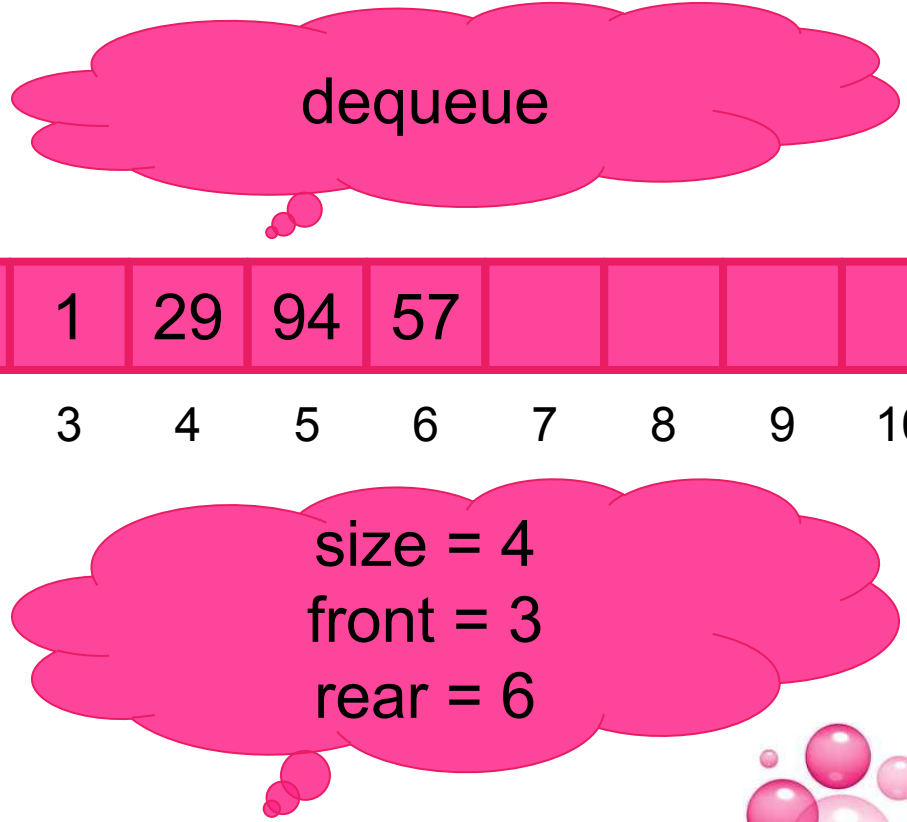
array implementation - addressing linear dequeue



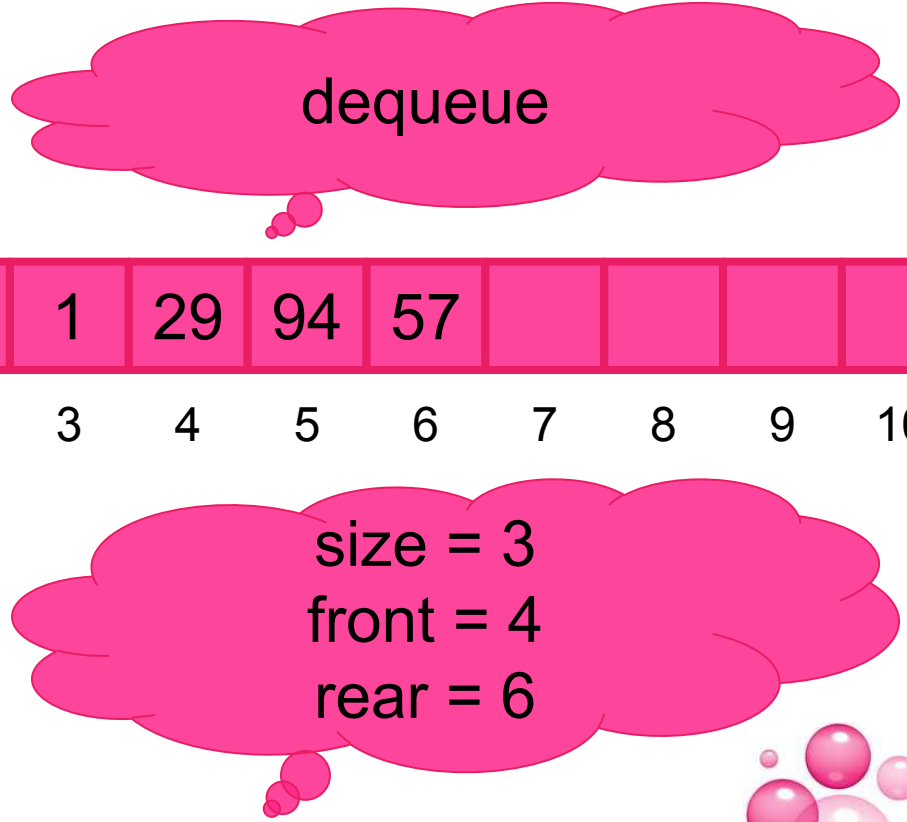
array implementation - addressing linear dequeue



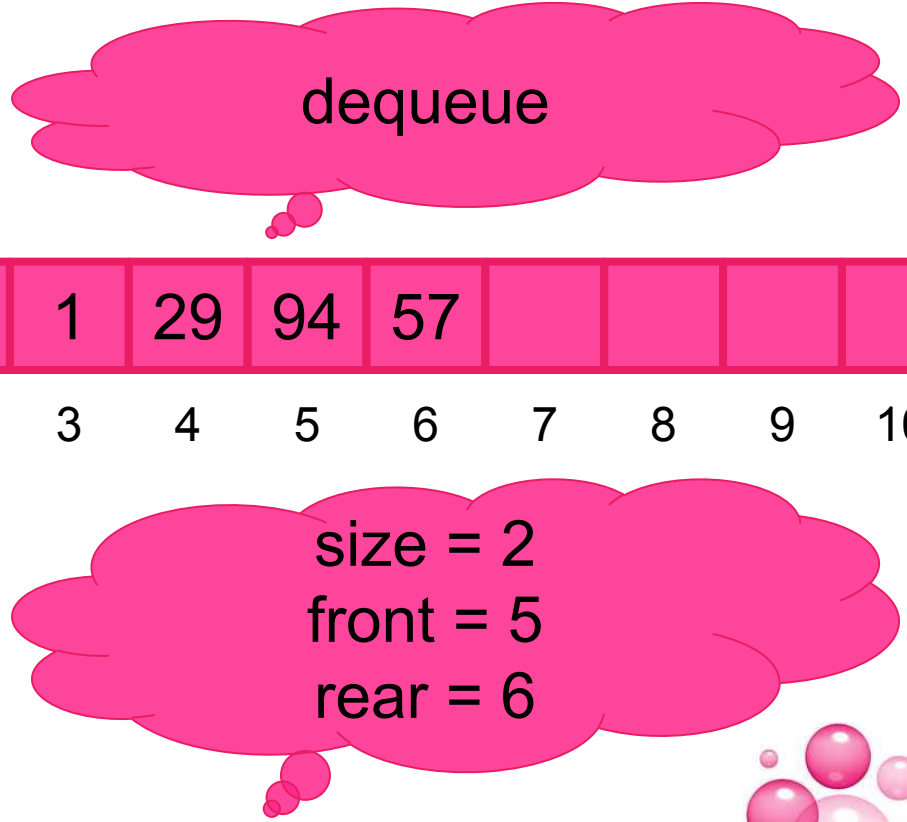
array implementation - addressing linear dequeue



array implementation - addressing linear dequeue



array implementation - addressing linear dequeue



array implementation - addressing linear dequeue

enqueue 36



0 1 2 3 4 5 6 7 8 9 10 11 12

front

size

size = 3
front = 5
rear = 7

array implementation - addressing linear dequeue

enqueue 78



0 1 2 3 4 5 6 7 8 9 10 11 12

front

size

size = 4
front = 5
rear = 8

array implementation - addressing linear dequeue

enqueue 61



0 1 2 3 4 5 6 7 8 9 10 11 12

front

size

size = 5
front = 5
rear = 9

array implementation - addressing linear dequeue

enqueue 80



0 1 2 3 4 5 6 7 8 9 10 11 12

front

size

size = 6
front = 5
rear = 10

array implementation - addressing linear dequeue

enqueue 5



0 1 2 3 4 5 6 7 8 9 10 11 12

front

size

size = 7
front = 5
rear = 11

array implementation - addressing linear dequeue

enqueue -4



0 1 2 3 4 5 6 7 8 9 10 11 12

front

size

size = 8
front = 5
rear = 12

array implementation - addressing linear dequeue

enqueue 13



0 1 2 3 4 5 6 7 8 9 10 11 12

front

size

size = 8
front = 5
rear = 12

array implementation - addressing linear dequeue

enqueue 13



0 1 2 3 4 5 6 7 8 9 10 11 12

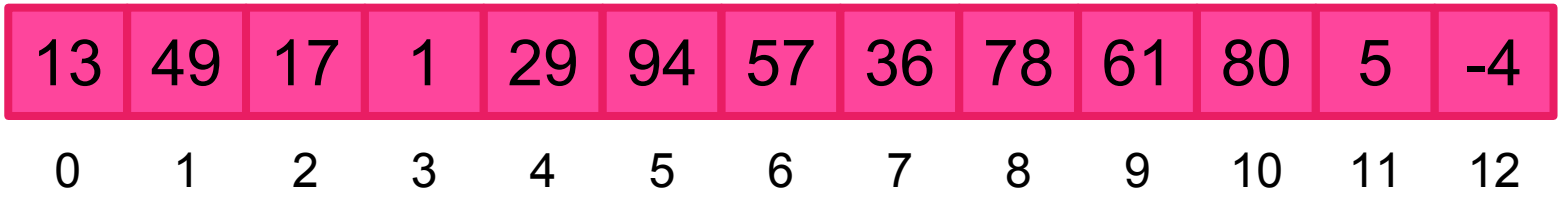
front

size

size = 9
front = 5
rear = 0

array implementation - addressing linear dequeue

enqueue 49



front

size

size = 10
front = 5
rear = 1

array implementation - addressing linear dequeue

enqueue 68

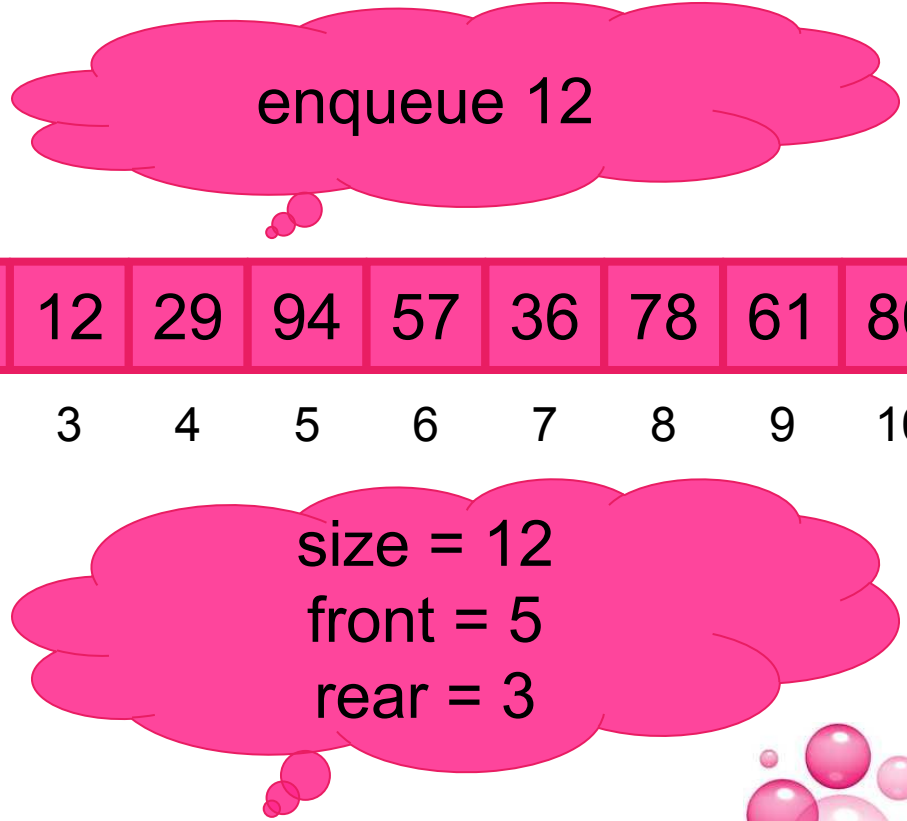


front

size

size = 11
front = 5
rear = 2

array implementation - addressing linear dequeue



array implementation - addressing linear dequeue

enqueue 11



front

size

size = 13
front = 5
rear = 4

array implementation - addressing linear dequeue

enqueue 25



0 1 2 3 4 5 6 7 8 9 10 11 12

front

size

size = 13
front = 5
rear = 4

array implementation - addressing linear dequeue

how to know if full



0 1 2 3 4 5 6 7 8 9 10 11 12

front

size

size = 13
front = 5
rear = 4

ciricular array implementation

operation	array
enqueue	O(1)
dequeue	O(1)
front	O(1)
rear	O(1)

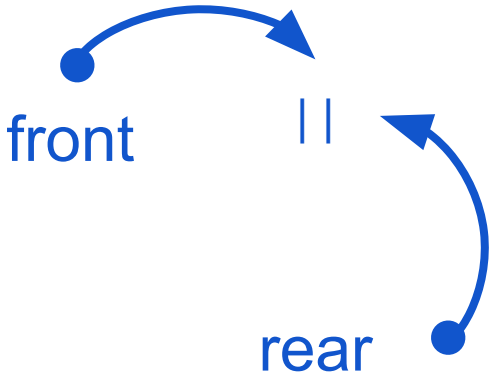


singly-linked list implementation

- though all operations run in constant time, space is not dynamic
- dynamic memory schemes
  - vector
    - similar to circular array
    - but how to do the copying?
  - singly-linked list
  - doubly-linked list



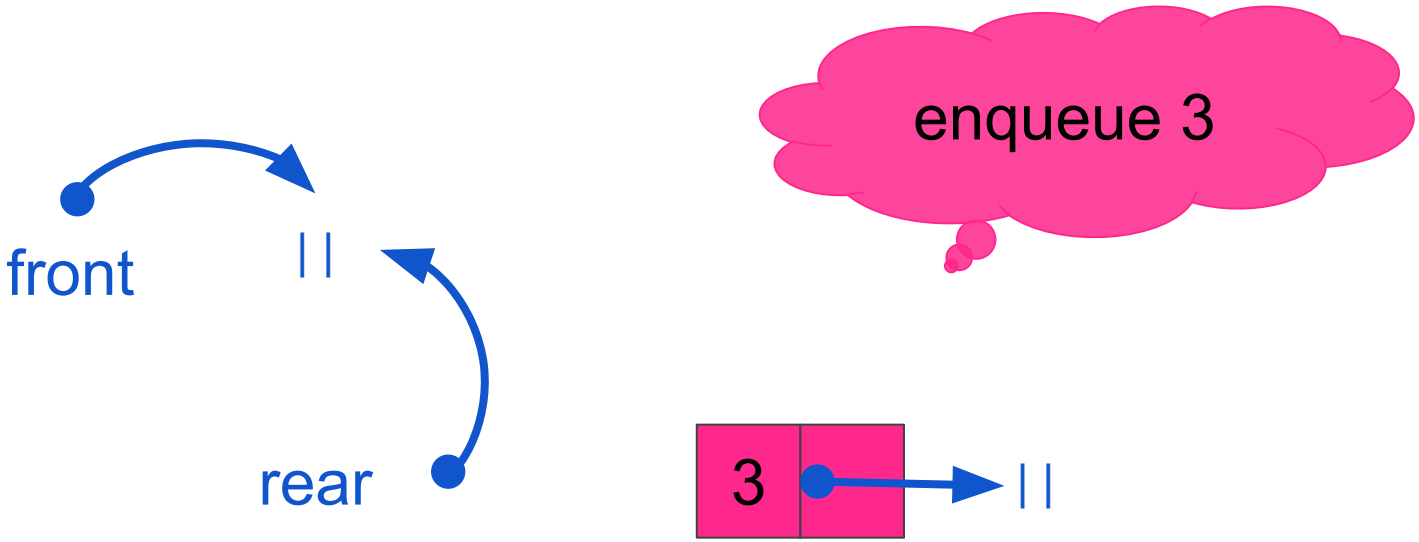
singly-linked list implementation



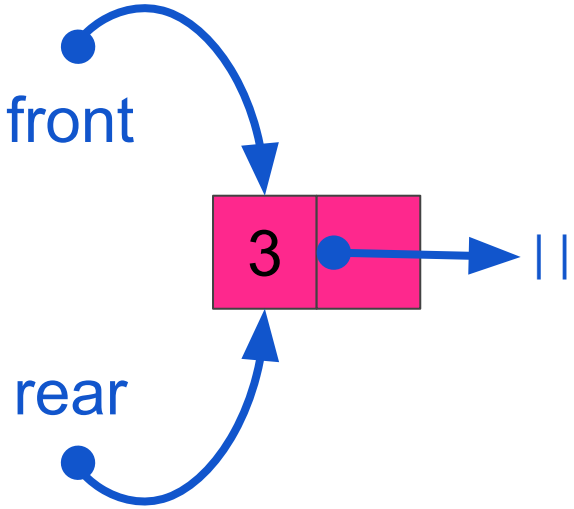


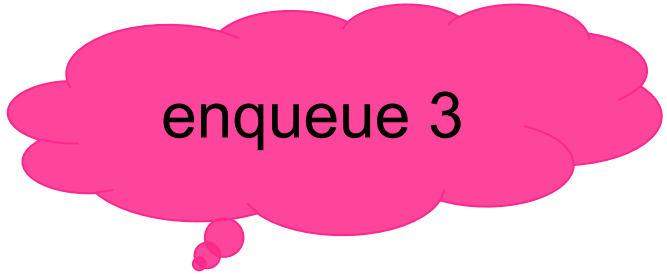


singly-linked list implementation

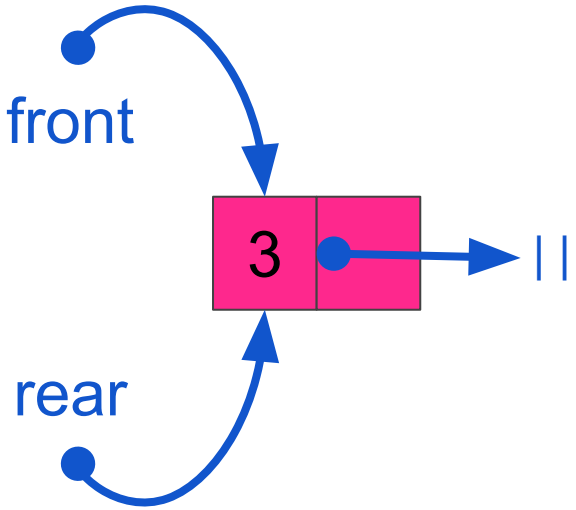


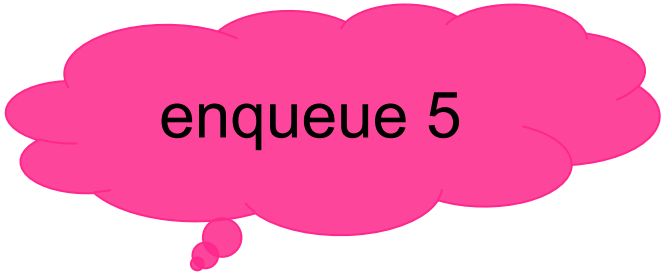
singly-linked list implementation

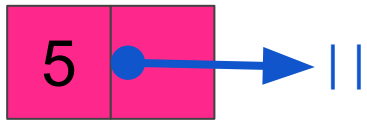




singly-linked list implementation

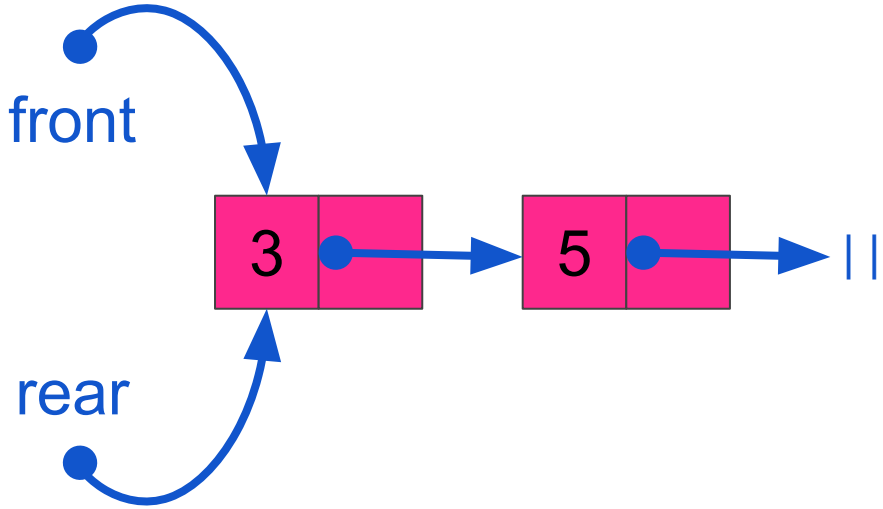


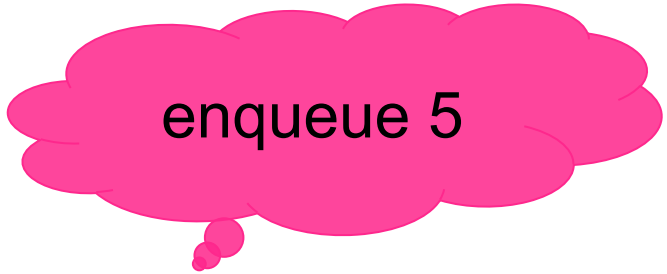




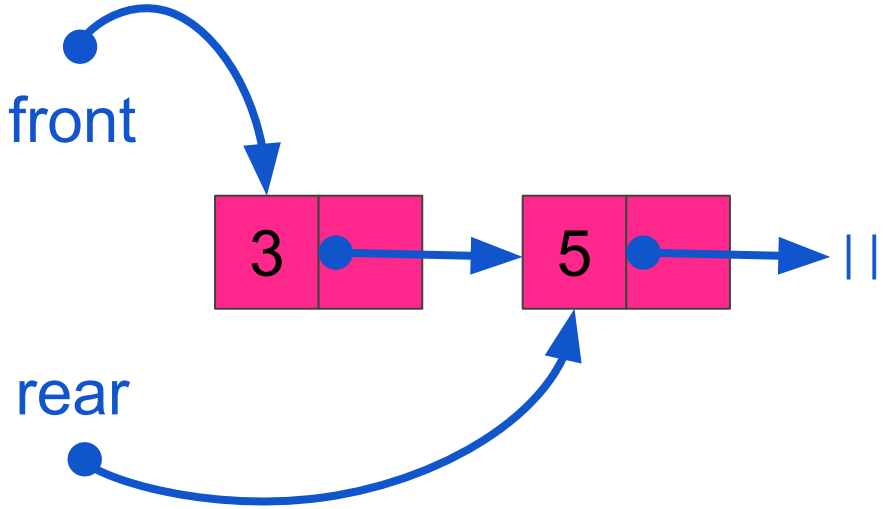


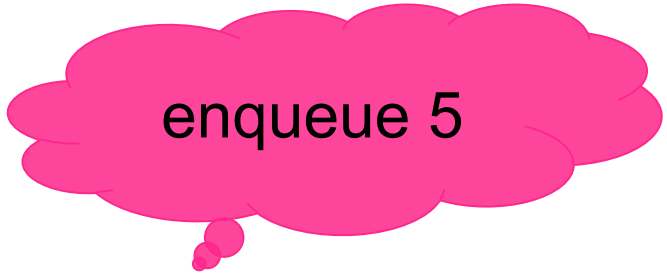
singly-linked list implementation



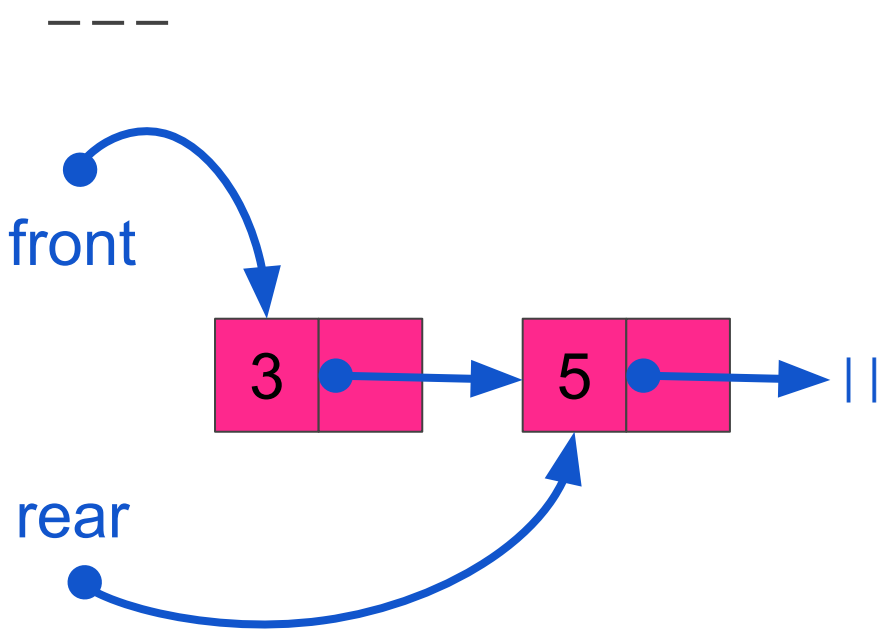


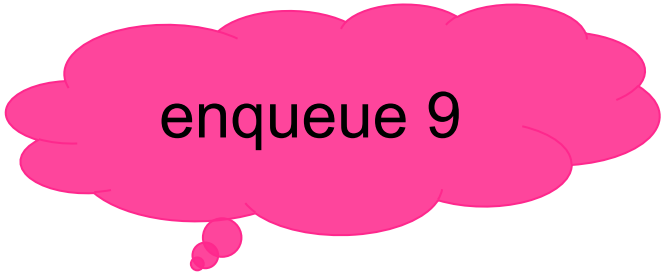
singly-linked list implementation

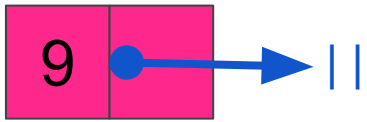




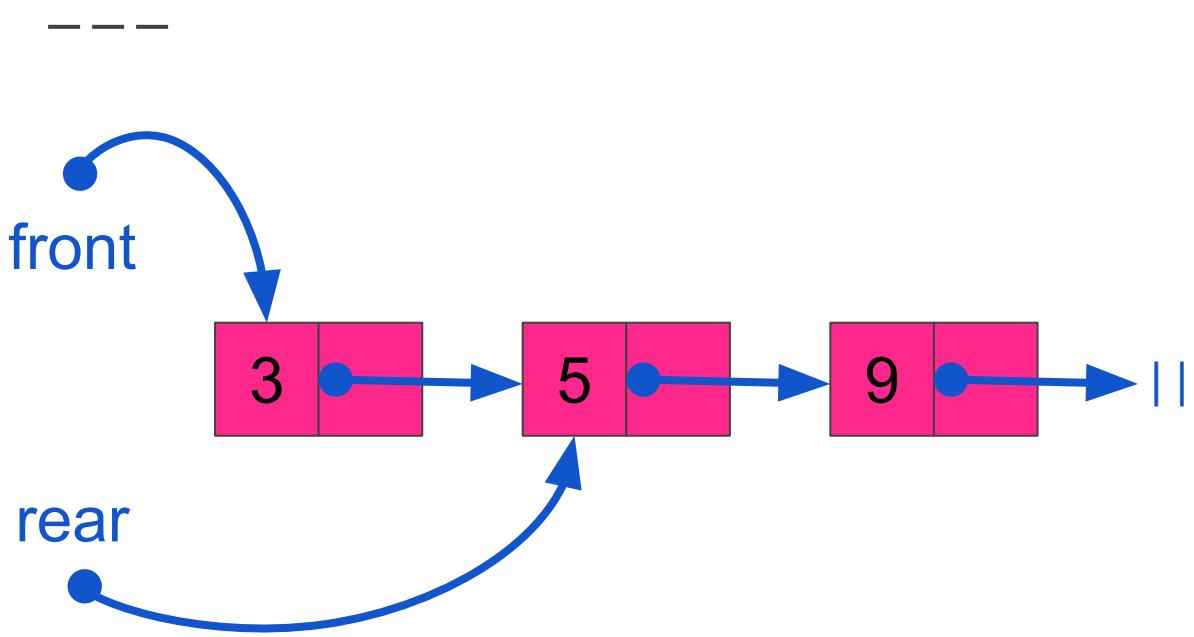
singly-linked list implementation

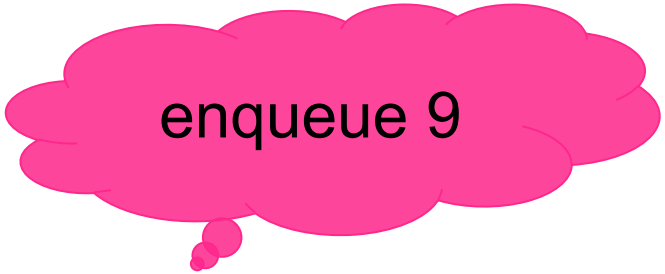




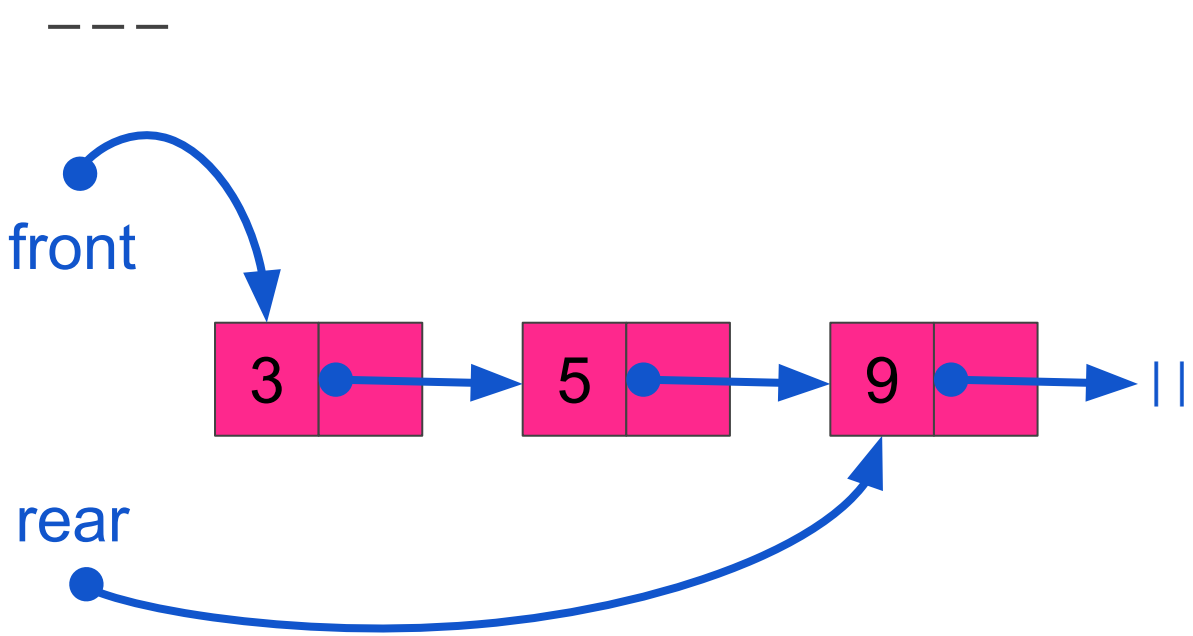


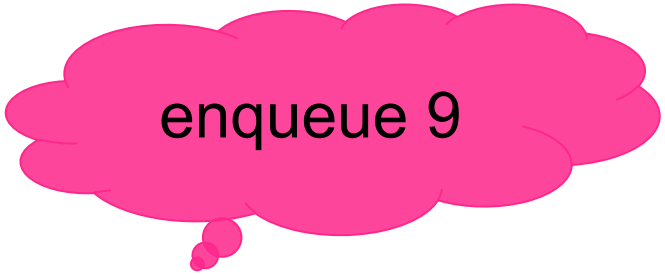
singly-linked list implementation



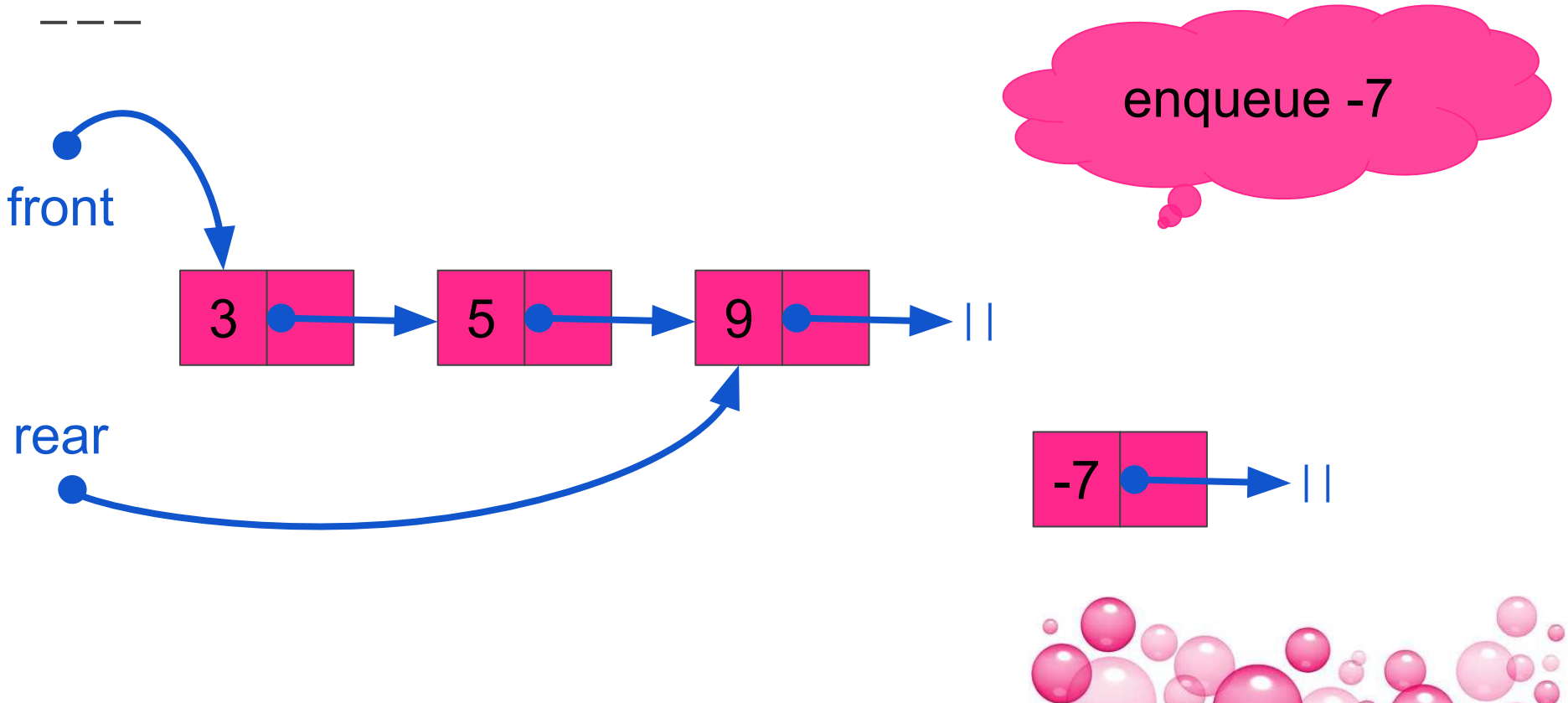


singly-linked list implementation

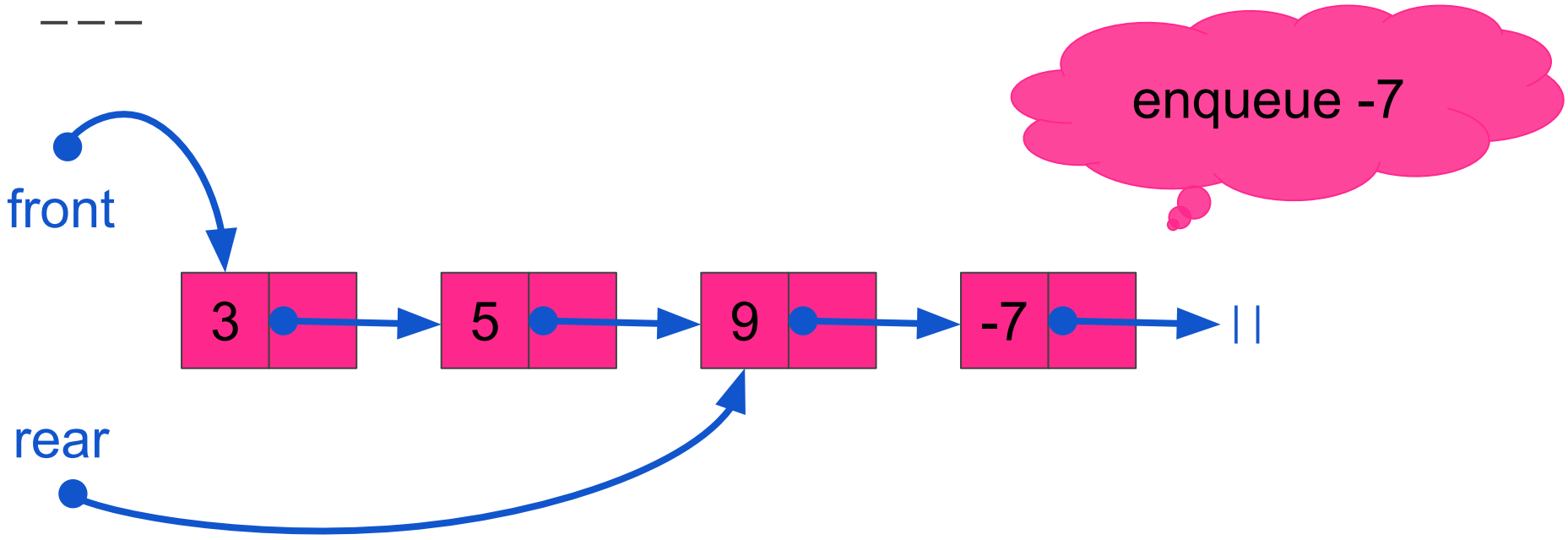




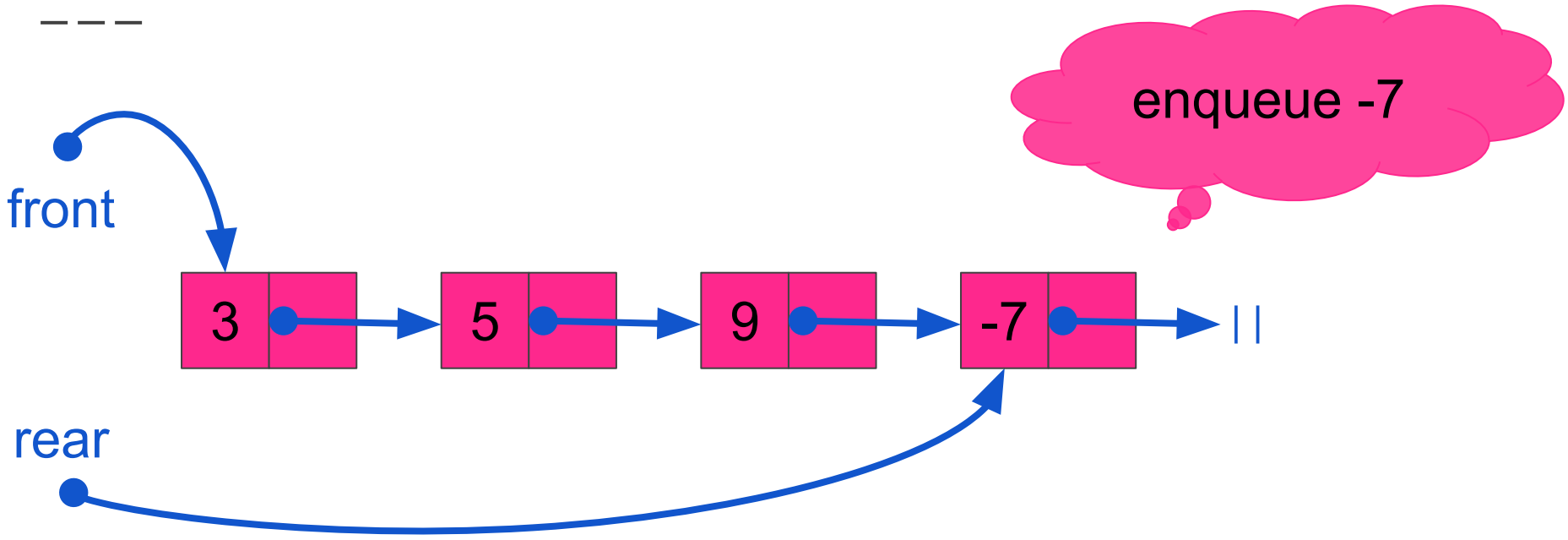
singly-linked list implementation



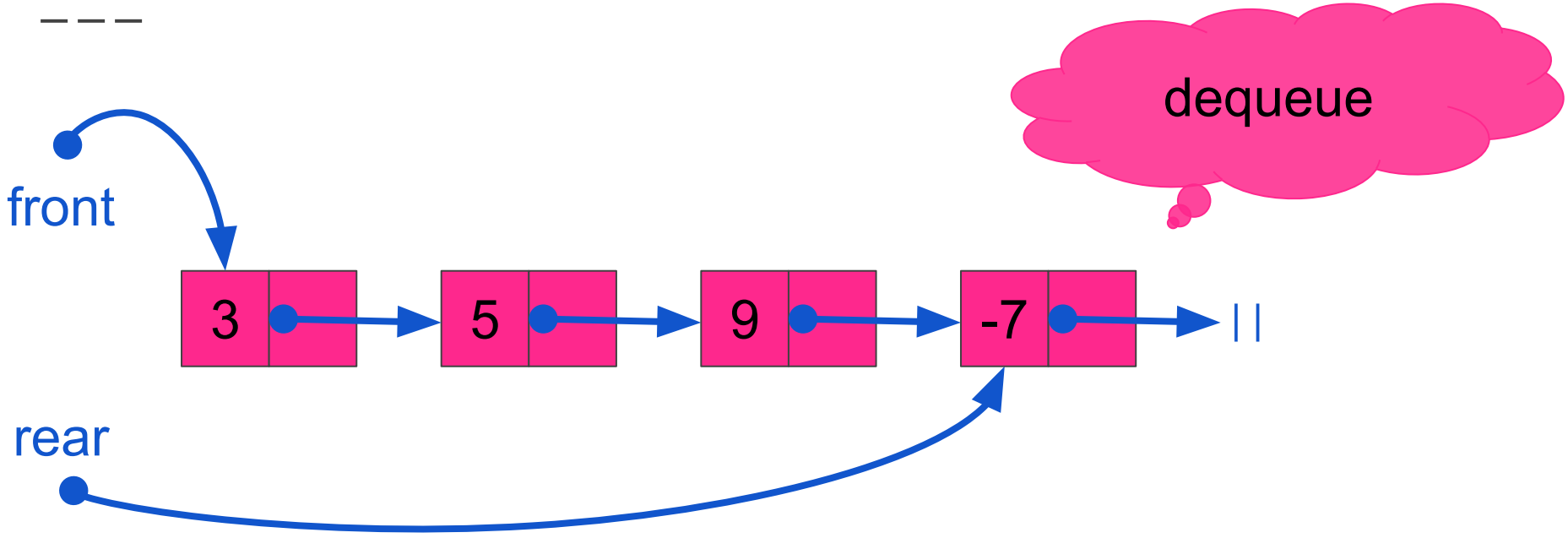
singly-linked list implementation



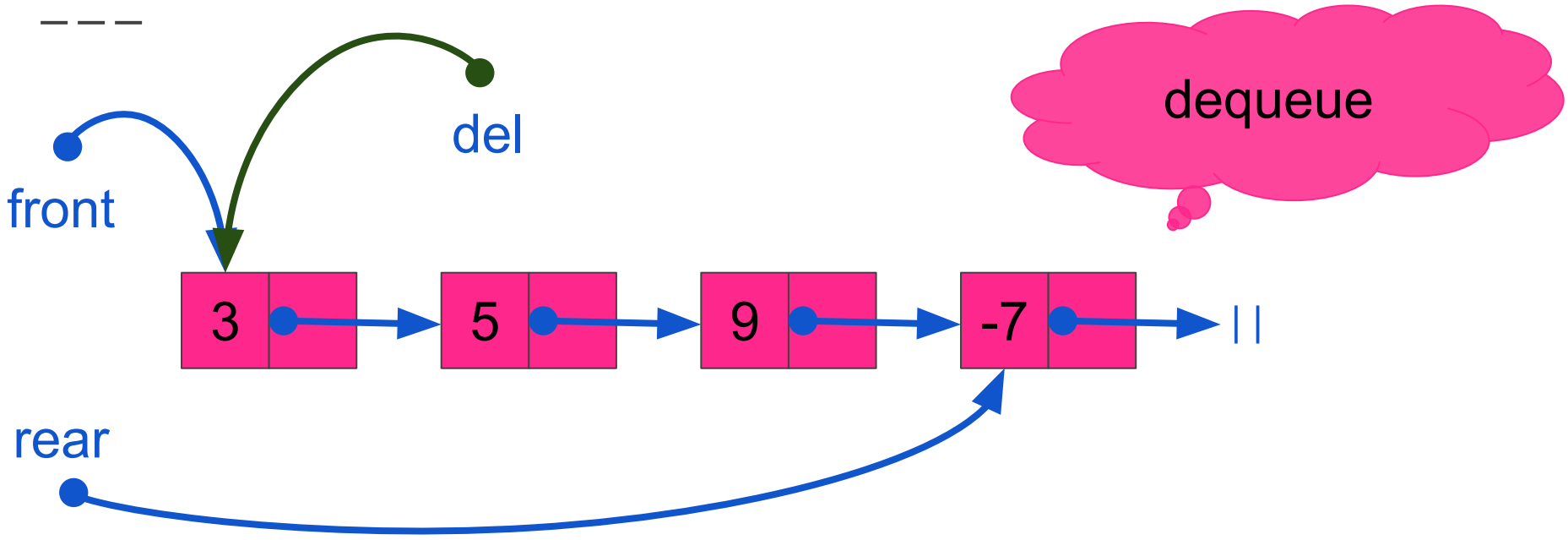
singly-linked list implementation



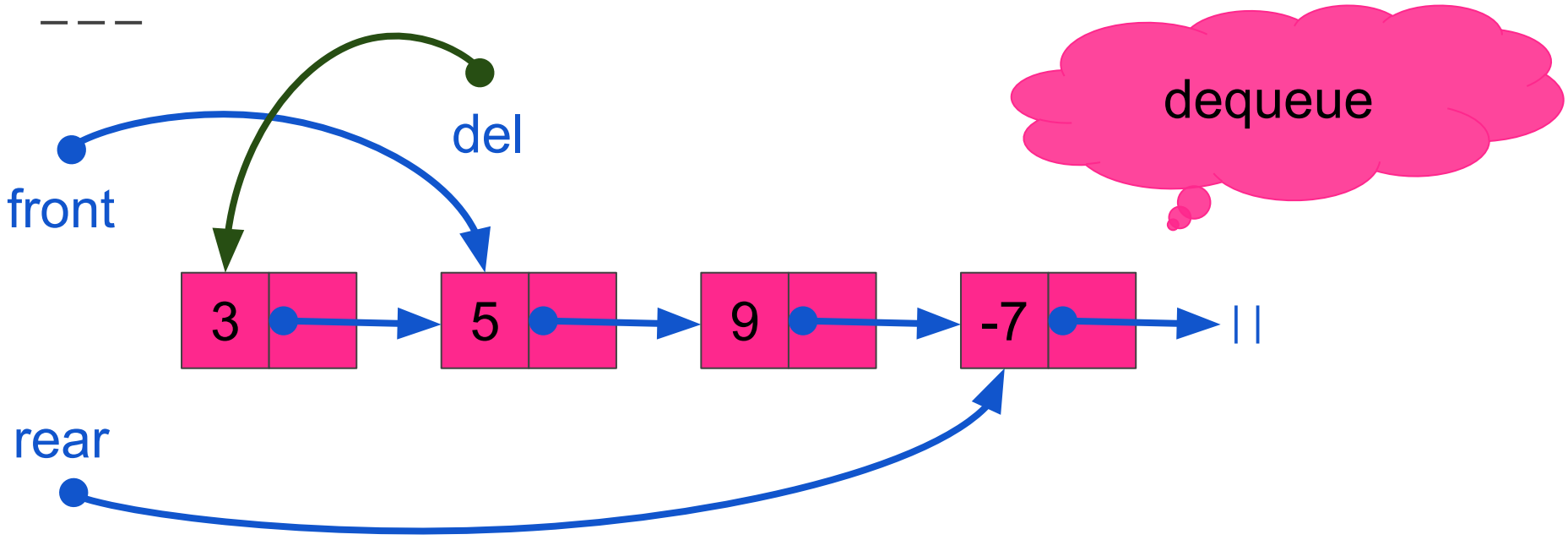
singly-linked list implementation



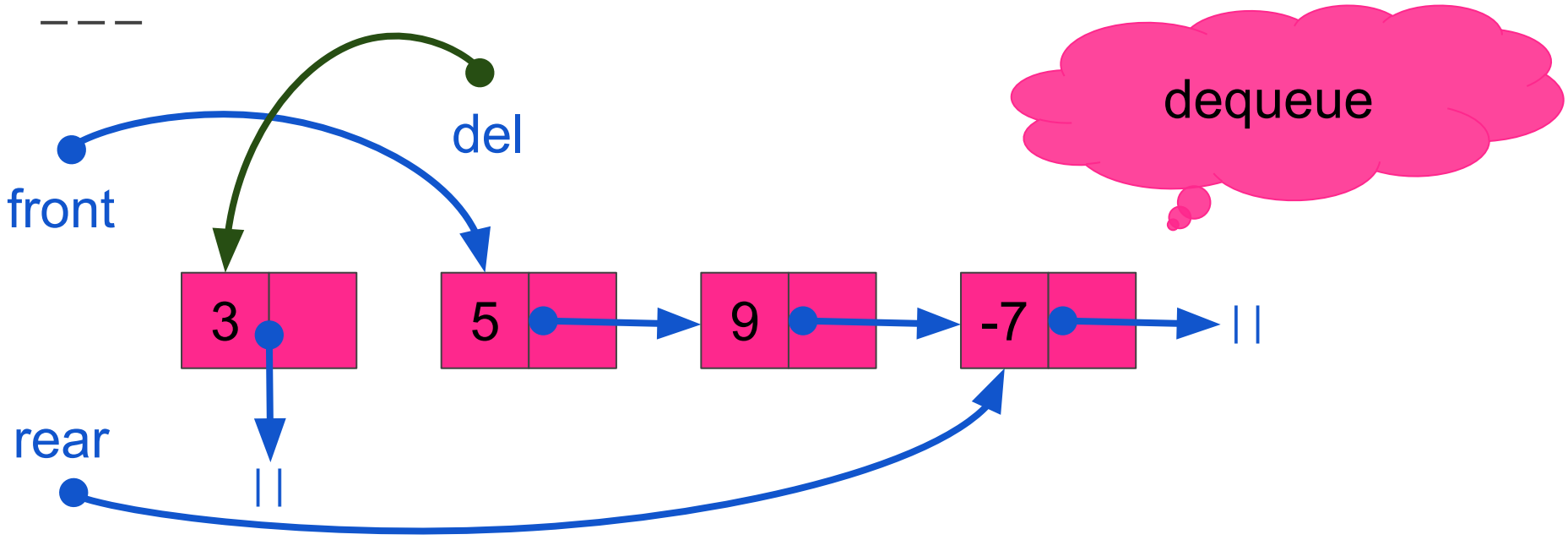
singly-linked list implementation



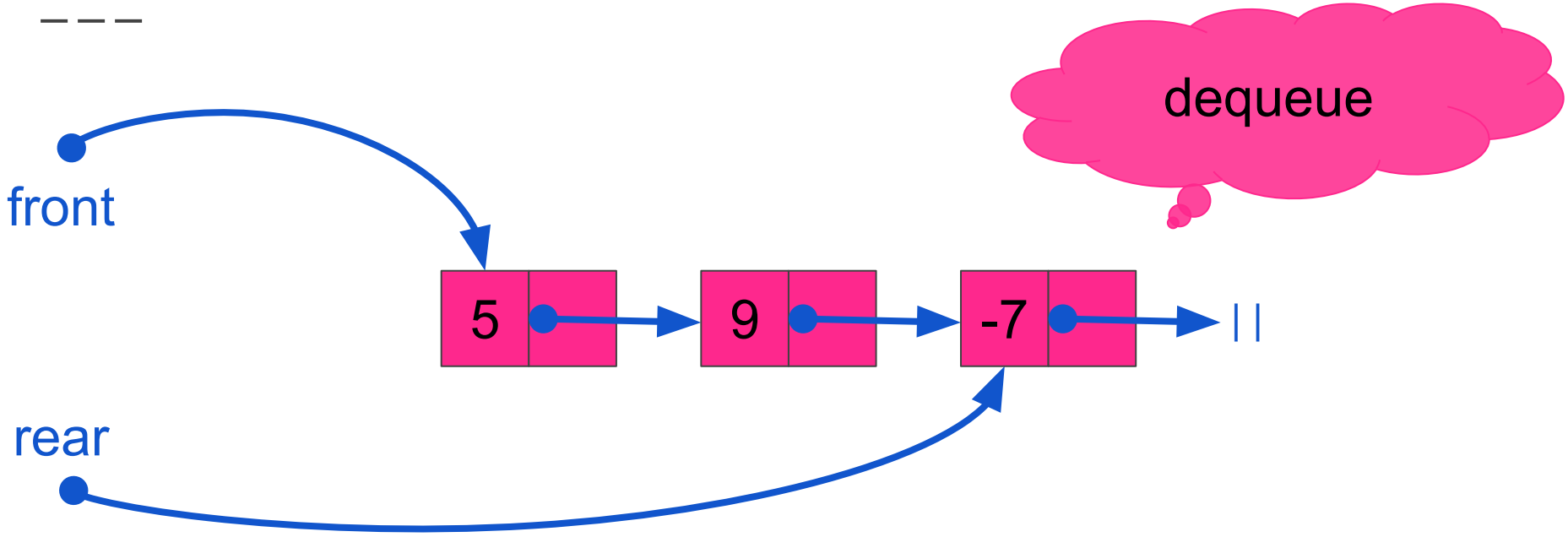
singly-linked list implementation



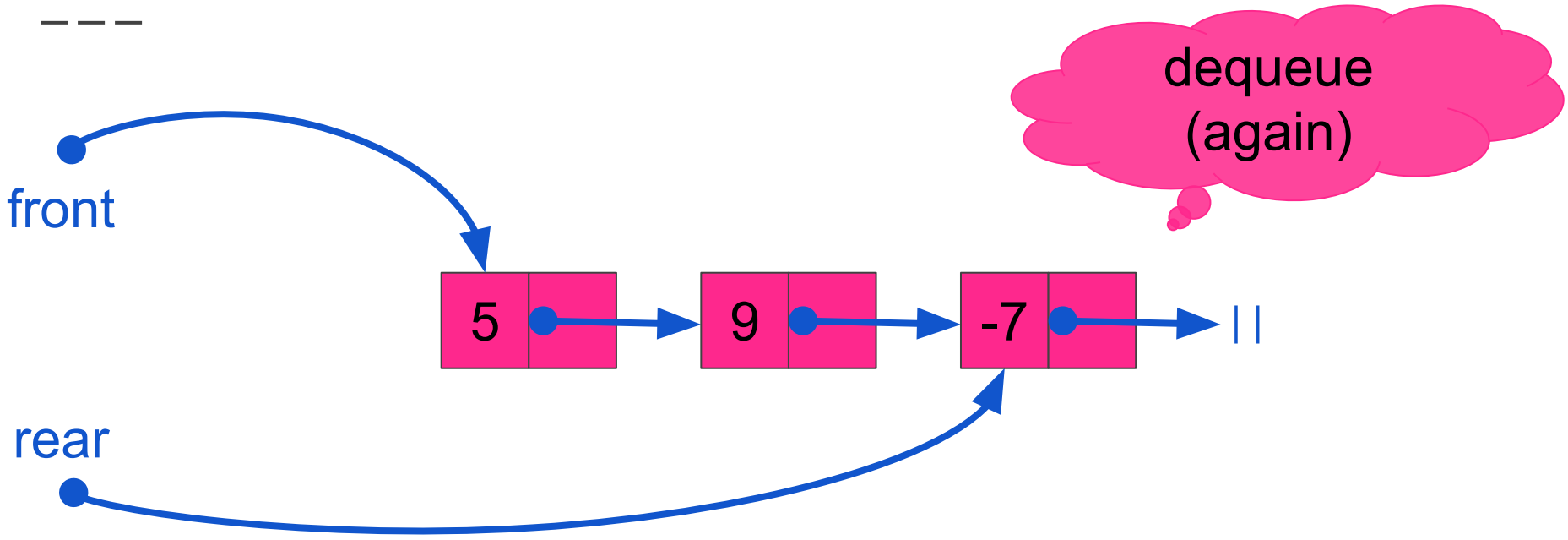
singly-linked list implementation



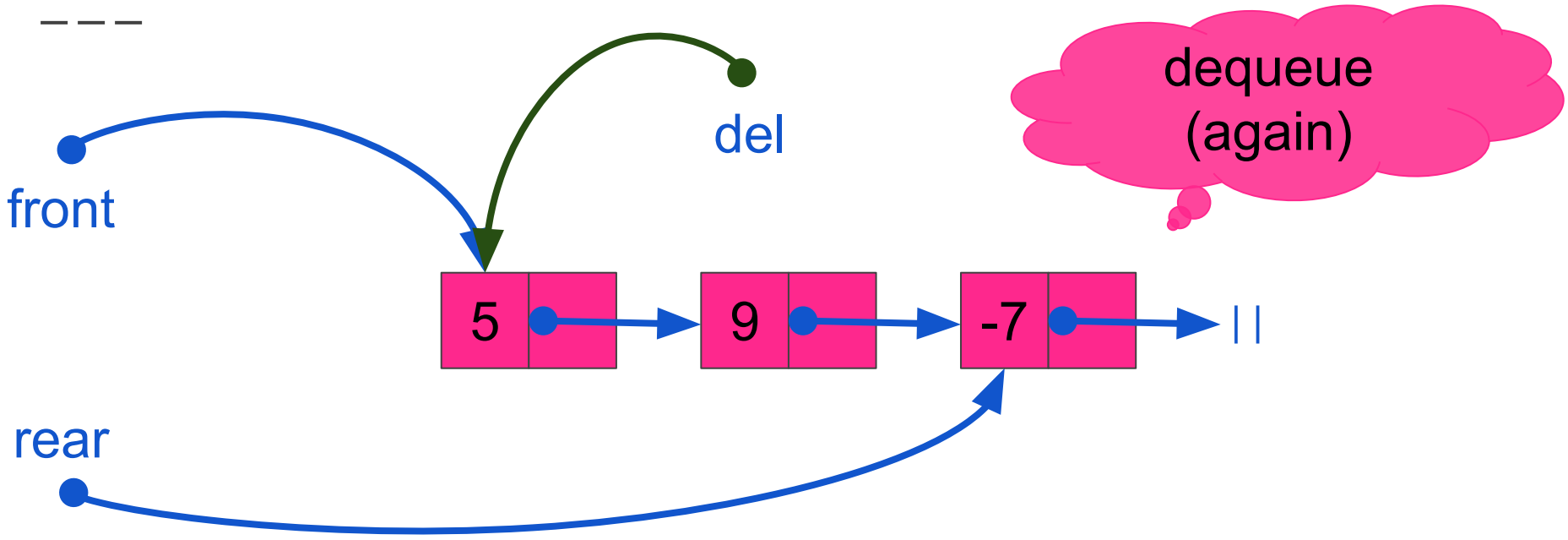
singly-linked list implementation



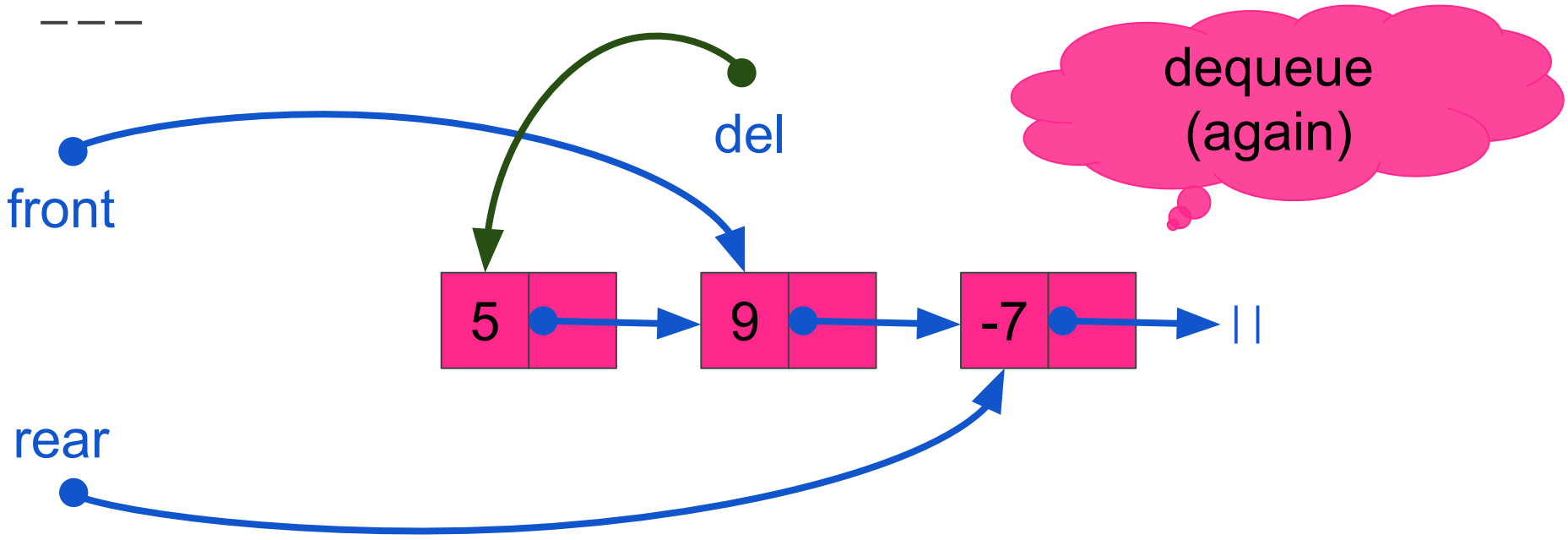
singly-linked list implementation



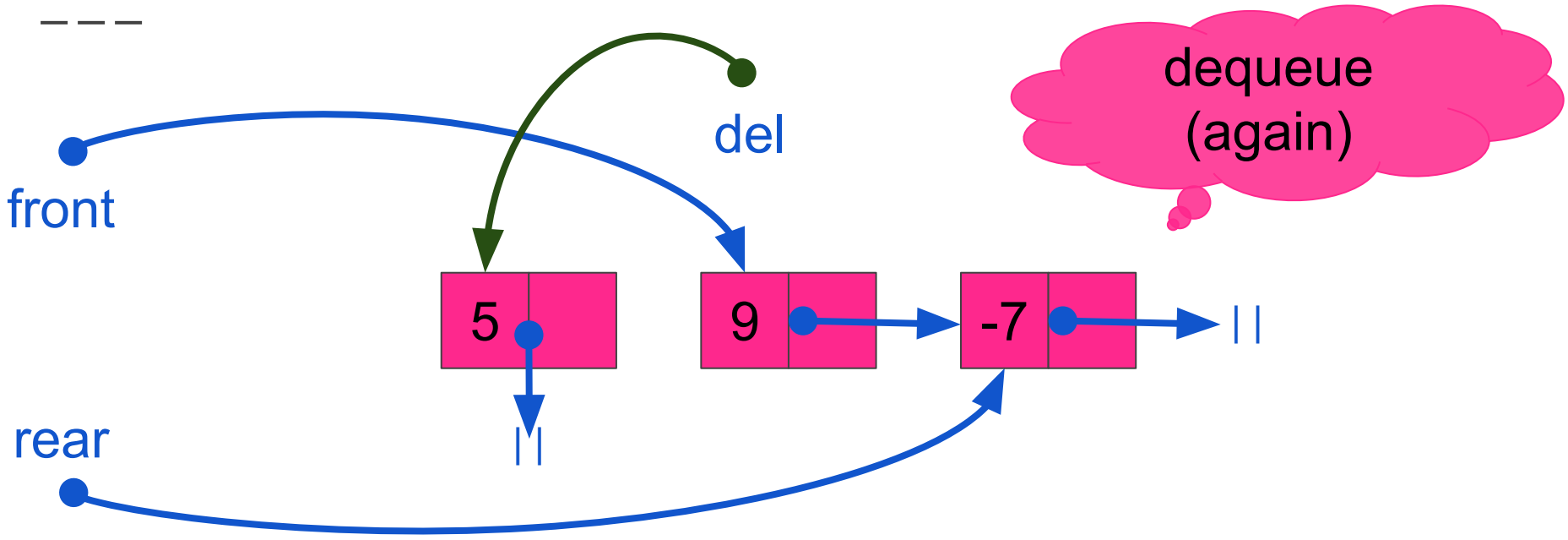
singly-linked list implementation



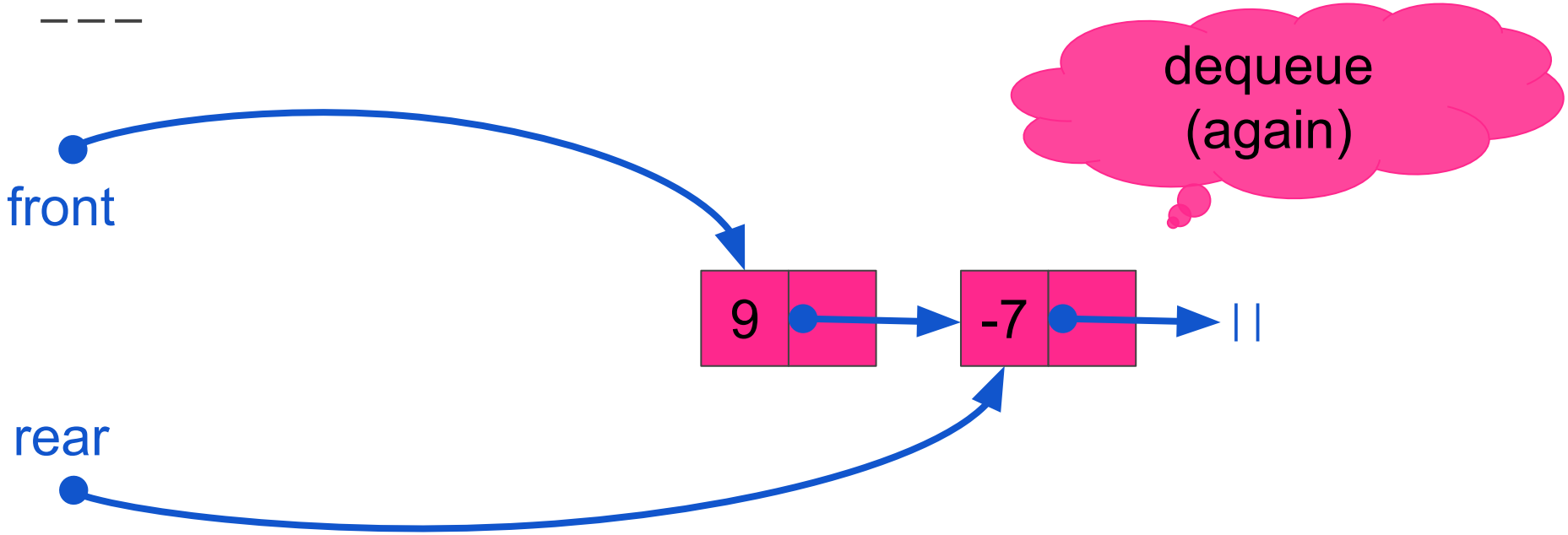
singly-linked list implementation



singly-linked list implementation



singly-linked list implementation



singly-linked list implementation

operation	array	singly-linked
enqueue	O(1)	O(1)
dequeue	O(1)	O(1)
front	O(1)	O(1)
rear	O(1)	O(1)

